arXiv:2405.20000v3 [math.NA] 9 Sep 2024

Manuscript submitted to XXX
Volume **X**, Number **0X**, 1 **2024** pp. **XX–XX**

COMBINING PHYSICS-INFORMED GRAPH NEURAL NETWORK AND FINITE DIFFERENCE FOR SOLVING FORWARD AND INVERSE SPATIOTEMPORAL PDES

Hao Zhang, Longxiang Jiang, Xinkun Chu, Yong Wen, Luxiong Li, Yonghao Xiao, Liyuan Wang*

[1]

Institute of Computer Application, China Academy of Engineering Physics, Mianyang, Sichuan, 621000, P. R. China.

Abstract. The great success of Physics-Informed Neural Networks (PINN) in solving partial differential equations (PDEs) has significantly advanced our simulation and understanding of complex physical systems in science and engineering. However, many PINN-like methods are poorly scalable and are limited to in-sample scenarios. To address these challenges, this work proposes a novel discrete approach termed Physics-Informed Graph Neural Network (PIGNN) to solve forward and inverse nonlinear PDEs. In particular, our approach seamlessly integrates the strength of graph neural networks (GNN), physical equations and finite difference to approximate solutions of physical systems. Our approach is compared with the PINN baseline on three well-known nonlinear PDEs (heat, Burgers and FitzHugh-Nagumo). We demonstrate the excellent performance of the proposed method to work with irregular meshes, longer time steps, arbitrary spatial resolutions, varying initial conditions (ICs) and boundary conditions (BCs) by conducting extensive numerical experiments. Numerical results also illustrate the superiority of our approach in terms of accuracy, time extrapolability, generalizability and scalability. The main advantage of our approach is that models trained in small domains with simple settings have excellent fitting capabilities and can be directly applied to more complex situations in large domains.

Keywords: physics-informed graph neural network, finite difference, nonlinear partial differential equations

1. Introduction.

PDEs are indispensable mathematical tools to describe the behavior of complex dynamical systems under varying conditions in multifarious science and engineering disciplines, such as thermodynamics, fluid mechanics, biochemistry, electronics, and materialogy. By studying PDEs, we can learn the evolutionary patterns of corresponding systems, in the meantime, understand the influence of key parameters of PDEs on the evolution process.

In scientific computing, there are three main issues related to PDE research: forward problem (solving PDEs), inverse problem and equation discovering problem.

* Corresponding author: wangly_xjtu@163.com .
[1]Emails: linusec@163.com, whutjlx@163.com, neocosmos@163.com, wenyong119321@163.com, m13368289607_1@163.com, xiao_yonghao@163.com.

In forward problems, governing equations are known, we need to find analytical or numerical solutions using conventional numerical methods or other surrogate methods. The inverse problem represents using observations to infer unknown parameters of the physical system, i.e., information of the governing equation is partially known. While discovering governing equation refers to the situation that the dynamical system has no analytic expressions, fortunately in these cases, observation data is abundant, and one needs to determine the causal factors or the potential governing equation from these observations. This paper focusses on the first two problems.

Except for a few simple PDEs that have analytical solutions, most of PDE problems need to be solved numerically. Decades of researches and efforts have been devoted to the investigation and development of numerical methods. At present, the mainstream numerical approaches include finite difference, finite element and finite volume method. Although the increasing computational power and capacity enable the computation of complex problems, numerical methods still suffer from enormous computational costs: long time consumption and resource burden. This issue becomes even more serious when solving complex inverse problems.

Recently machine learning (ML), especially deep learning (DL) technology, emerges as a promising direction for solving scientific problems, such as exploring physical systems that are not yet fully understood and accelerating simulation process, due to its powerful feature extraction ability and computation efficiency. Nevertheless, because of their black-box nature, applying DL model directly is insufficient to solve complex scientific problems. The research community is beginning to integrate the scientific knowledge and DL models for the purpose of alleviating the shortage of training data, increasing models' generalizability and ensuring the physical plausibility of results.

In the past few years, integrating scientific knowledge with DL models has been a research boom. The primitive idea can be traced back to Raissi et al. [1]. They proposed a novel framework of neural network named PINN, which successfully integrates the underlying physical law described by PDEs with fully connected (FC) networks. This work lays a solid foundation for solving both forward and inverse PDEs and has inspired plenty of extended studies. The general principle of PINN is to take the physical information as constraint and incorporate it into the loss function to steer the deep model to output physically consistent predictions. The popularity of PINN is mainly attributed to three aspects: the flexibility of meshless, the simplicity of the neural network, and the automatic differentiation technique. PINN achieves great success in many applications fields, including fluid mechanics [2, 3, 4, 5], medical diagnosis [6, 7], materials science [8, 9], engineering mechanics [10, 11]. Despite the success of PINN, it still suffers from some limitations. On the one hand, PINN lacks the ability to generalize to out-of-sample scenarios, such as varying time scales, spatial resolutions and computational domains. On the other hand, PINN requires a large number of collocation points to calculate PDE residuals, which can lead to a huge training cost, especially in high-dimensional space. Based on PINN, a lot of extension works have been done to improve the performance of PINN, such as synthesizing with numerical methods [12], defining problem-specific activation functions [13], optimizing the sampling of collocation points [5, 14, 15], and balancing residuals with adaptive weights [12, 14, 16, 17, 18], configuring special neural network frameworks [19, 20, 21].

In addition to physics-based models, data-driven methods [22, 23, 24] also seek nonlinear mappings from parametric DNNs to numerical solutions. In 2020, a Google research team [25] applied GNN to particle simulation, where dynamics between particles are simulated through a message passing mechanism. Although this method achieves the best simulation accuracy at the time, its computational efficiency is low due to its neighborhood calculation method. Later, this team [26] further expanded their work by applying GNNs to mesh-based simulations and proposed the MeshGraphNets network. By evaluating a wide range of physical systems, this method was shown to predict dynamics of underlying systems accurately and efficiently, and was capable of learning resolution adaptive dynamics and scalable to more complex spaces.

As mentioned above, based on existing advances in the physics-informed DL community, the goal of this paper is to construct a surrogate model for solving spatiotemporal PDEs in irregular domains. This model learns mesh-based solutions of PDEs with a few or no pre-computed labeled data by integrating physical information into GNN with acceptable accuracy and generalizability while substantially reducing the computational cost. We are interested in dealing with unstructured mechanisms rather than regular grids, as in practice, the computational domain of complex physical systems is usually irregular. Mesh is a powerful tool for appropriately representing irregular geometry, and employing mesh representation to solve PDEs supports adaptive resolutions and achieves a favorable tradeoff between accuracy and efficiency. Based on the mesh generated by numerical methods, by establishing an one-to-one mapping between mesh points and graph nodes, we can describe unstructured mesh data with graphs. Therefore, we naturally use GNNs instead of other neural networks. GNN can not only model the structure between nodes, but also perform local information mining and relational reasoning based on the message passing mechanisms, which implicitly introduces inductive biases that is conductive to the learning process.

In a word, we propose a novel PIGNN framework, combined with a finite-difference-based method for computing differential operators on the graph, so as to accurately and effectively solve the forward and inverse PDEs in mesh space. This work provides an important step towards advancing computation of solving PDEs. Compared to the existing literature, our main contributions are as follows:

1) We introduce a novel PIGNN framework to efficiently solve the forward and inverse PDEs. For the forward problem, the computation of physics-informed loss requires no labeled data. While for the inverse problem, regularizations constrained by underlying PDEs and few labeled data narrow the search space of parameters, which makes it possible for the model to rapidly learn unknown parameters and solutions with ensured physical consistency during optimization.

2) We propose a finite-difference-based method combined with the ordinary least square technique to approximate the differential operators defined on the graph. At the same time, we theoretically discuss the approximation error of differential operators.

3) The proposed PIGNN is directly constructed from the unstructured mesh, providing the ability to handle irregular domains. Capitalizing on the powerful mesh representation and learning with certain resolution, our method can generalize to scenarios with refined or coarse resolution (i.e., other discretization schemes) not seen during training.

4) We present general rules of selecting appropriate features for nodes and edges, which are found to be an important factor in enabling the model to better generalize to out-of-sample scenarios.

5) A series of numerical experiments are conducted on several typical PDEs to quantitatively evaluate our framework. Numerical results show that our method outperforms the baseline model in terms of accuracy, time extrapolation, generalization and scalability. Most importantly, our model trained in small domains with simple settings has excellent fitting ability, and can be directly applied to complex settings with large domains, which further sheds lights on the superiority of the proposed PIGNN framework.

6) Compared with [27], the main difference of this paper is that we apply PIGNN to inverse problems and extend PIGNN to distributed multi-GPU systems to solve large-scale problems, so as to show the scalability of PIGNN. Meantime, the theoretical error estimation of differential operators is presented.

The rest of this paper is organized as follows. In Section 2, we elaborate on the PIGNN framework, including the problem setup, model architecture, physics-informed loss function and general rules of selecting features. Section 3 illustrates some computational issues defined on the graph. We first present the estimation method for differential operators on the mesh, then state the interpolation method involved in the inverse problem. Section 4 provides some numerical experiments to illustrate the outstanding performance of our PIGNN. Finally, Section 5 concludes the paper.

2. **PIGNN Framework.**

2.1. **Problem setup.**

We consider a dynamical system in a bounded domain $\Omega \subset \mathbb{R}^d$ governed by a system of nonlinear PDEs of the following general form,

$$\frac{\partial \mathbf{u}(\mathbf{x},t)}{\partial t} + \mathcal{F}\big[\mathbf{u}, \nabla_{\mathbf{x}}\mathbf{u}, \Delta_{\mathbf{x}}\mathbf{u}, \cdots; \boldsymbol{\lambda}\big] = \mathbf{f}(\mathbf{x},t), \qquad \mathbf{x} \in \Omega,\, t \in \mathbb{T} = [0,T], \tag{1}$$

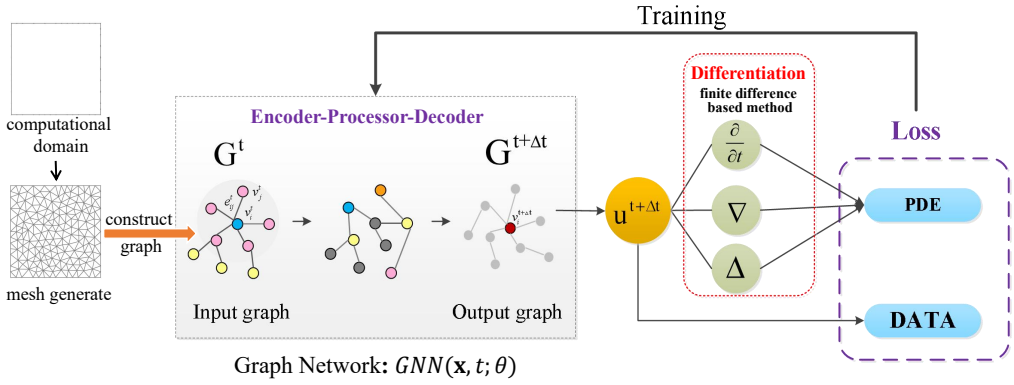
$$\mathcal{B}\big(\mathbf{u}(\mathbf{x},t)\big) = b(\mathbf{x},t), \qquad \mathbf{x} \in \partial\Omega,\, t \in \mathbb{T}, \tag{2}$$

$$\mathcal{I}\big(\mathbf{u}(\mathbf{x},0)\big) = i(\mathbf{x}), \qquad \mathbf{x} \in \Omega,\, t = 0. \tag{3}$$

where the system state $\mathbf{u}(\mathbf{x},t) \in \mathbb{R}^n$ denotes the solution of PDEs evolved over time horizon $t \in \mathbb{T}$ and spatial locations $\mathbf{x} \in \Omega$, T is the terminal time, $\frac{\partial \mathbf{u}}{\partial t}$ is the first-order time derivative term, $\mathcal{F}[\cdot\,;\boldsymbol{\lambda}]$ is a complex nonlinear functional of state variable $\mathbf{u}(\mathbf{x},t)$ and its spatial derivatives, parameterized by $\boldsymbol{\lambda}$. Here, ∇ and Δ denote the gradient and Laplace operators, respectively. $\mathbf{f}(\mathbf{x},t)$ is the source term ($\mathbf{f} = \mathbf{0}$ represents no extra source input to the system). In addition, the system of PDEs is subjected to BCs and ICs, defined by Eq. (2)-(3), where $\partial\Omega$ denotes the boundary of spatial domain. Such PDEs describe a wide range of propagative physical systems, including heat dissipation, disease progression, cellular kinetics, fluid dynamics and so on.

In this paper, our objective is to propose an innovative PIGNN paradigm to establish a solution method for the above PDE system in both forward and inverse settings. In the forward setting, given specific ICs, BCs and PDE parameter $\boldsymbol{\lambda}$, we aim to find solutions for PDEs. The physics-informed loss function is constructed

based on the residuals of PDEs, and the overall learning process is unsupervised. While for the inverse problem, ICs and BCs are given, $\boldsymbol{\lambda}$ is unknown, but a few observations of the system state are available. What we need to do is to determine the system response as well as identify the unknown parameter $\boldsymbol{\lambda}$ that best fits the observations. The working mechanism of PIGNN is shown in Fig. 1. In general, we can interpret PIGNN as a neural solver for PDEs. The prior knowledge of physics is incorporated into PIGNN through specially designed loss functions to capture the dynamic patterns.



PDE Residual: $\mathcal{R}(\mathrm{x}, t+\Delta t; \boldsymbol{\theta}, \boldsymbol{\lambda}) = \dfrac{\mathbf{u}(\mathrm{x}, t+\Delta t; \boldsymbol{\theta}) - \mathbf{u}(\mathrm{x}, t)}{\Delta t} + \mathcal{F}(\mathbf{u}(\mathrm{x}, t+\Delta t; \boldsymbol{\theta}), \nabla_{\mathrm{x}}\mathbf{u}(\mathrm{x}, t+\Delta t; \boldsymbol{\theta}), \cdots; \boldsymbol{\lambda}) - \mathrm{f}(\mathrm{x}, t+\Delta t) \to 0$

Loss Function: $\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\lambda}) = (1-\gamma)\mathcal{L}_{PDE}(\boldsymbol{\theta}, \boldsymbol{\lambda}) + \gamma \mathcal{L}_{DATA}(\boldsymbol{\theta})$

where: $\mathcal{L}_{PDE}(\boldsymbol{\theta}, \boldsymbol{\lambda}) = \mathbb{E}[\|\mathcal{R}(\mathrm{x}, t+\Delta t; \boldsymbol{\theta}, \boldsymbol{\lambda})\|_2]$,
$\mathcal{L}_{DATA}(\boldsymbol{\theta}) = \mathbb{E}[\|\mathbf{y}(\mathrm{x}, t+\Delta t) - \mathbf{u}(\mathrm{x}, t+\Delta t; \boldsymbol{\theta})\|_2]$.

Solution: $\{\boldsymbol{\theta}^*, \boldsymbol{\lambda}^*\} = \arg\min_{\{\boldsymbol{\theta}, \boldsymbol{\lambda}\}} \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\lambda})$

FIGURE 1. The schematic diagram of PIGNN for solving forward and inverse problems. Unstructured meshes are generated using numerical methods. The Encoder component of the network constructs the input graph from mesh data, Processor updates and transmits the information, and Decoder outputs PDE solutions. By applying the finite-difference-based method, we compute all differential terms and incorporate prior knowledge of PDEs into the network via the specific PDE loss.

Simulation of PDE dynamics relies on appropriate spatial-temporal discretizations. Our first step is to discretize the computational domain Ω by constructing a simulation mesh on the domain surface, which can be implemented by mature mesh generation algorithms, such as Delaunay triangulation, advancing front method, and mapping method. At time t, let $\mathcal{M}^t = (\mathcal{V}, \mathcal{E}^M)$ be an unstructured mesh with mesh nodes $\mathcal{V}$ connected by mesh edges $\mathcal{E}^M$. $N = \|\mathcal{V}\|$ denotes the number of mesh nodes. Each node $i \in \mathcal{V}$ is associated with a coordinate $\mathbf{x}_i$ in the mesh space and a feature vector $\boldsymbol{f}_i$ describing the system state. By constructing an one-to-one mapping between mesh nodes and graph nodes, the unstructured mesh can be naturally represented as a graph. Let $G^t = (V, E^G)$ be the undirected graph, where $V = \{\mathbf{x}_i\}_{i=1}^N$ is the set of graph nodes, and the coordinate of the i-th node on the graph is still represented by its mesh space coordinate because of the one-to-one mapping. $E^G = \{\boldsymbol{e}_{ij}\}_{i,j \in V}$ denotes the set of graph edges, where the directional edge $\boldsymbol{e}_{ij}$ stands for the connection from node i (sender) to node j (receiver). Each

node and edge is associated with a node feature vector and an edge feature vector, separately. By abuse of notation, we use the same symbols for edge features $E^G = \{\boldsymbol{e}_{ij}\}_{i,j\in V}$ and node features $V = \{\boldsymbol{v}_i\}_{i=1}^N$ on the graph. $\boldsymbol{e}_{ij} = \{\mathbf{x}_{ij}, \|\mathbf{x}_{ij}\|\}$ represents the edge feature vector, which in our setting contains the relative displacement vector between node i and j: $\mathbf{x}_{ij} = \mathbf{x}_i - \mathbf{x}_j$ and its Euclidean distance $\|\mathbf{x}_{ij}\|$. $\boldsymbol{v}_i$ denotes the feature vector of the i-th node, including the learned node solution $\mathbf{u}(\mathbf{x}, t)$, a 2-dimensional one-hot vector indicating the node type, and other physical variables providing additional spatiotemporal information. So far, solutions of the PDE system have been successfully discretized by these graph nodes. Let $\mathbf{u}(\mathbf{x}_i, t)$ be the i-th node solution of PDEs at time t. Our goal now is to find node solutions to the PDE system Eq. (1)-(3) on the graph for a time range after a certain initial time.

2.2. **Model architecture.**

Note that the current time t is equal to the current time step multiplied by the time interval. We use the term time step to represent the time variable in the rest of this article for convenience. Assuming that $F : \mathbb{R}^n \to \mathbb{R}^n$ models the dynamics of solutions from the current time step t to the next time step $t + \Delta t$, where Δt is the time interval. In other words, at each time step, mesh-based solutions of system Eq. (1)-(3) are computed iteratively in a rollout manner by, $\mathbf{u}(\mathbf{x}_i, t + \Delta t) = F\big(\mathbf{u}(\mathbf{x}_i, t)\big)$. Our task then is, given the graph at time step $t : G^t$, to learn a model that determines dynamic quantities of the graph at the next time step $t + \Delta t$. We consider a general type of GNN proposed by Gilmer et al. [28] to model variations of solutions from the previous time step to the consequent time step, namely, $F\big(\mathbf{u}(\mathbf{x}_i, t)\big) = \mathbf{u}(\mathbf{x}_i, t) + GNN(\cdot; \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ denotes the parameters of GNN. Our PIGNN adopts the same Encoder-Processor-Decoder architecture as that in Sanchez-Gonzalez et al. [25, 26], which is shown in Fig. 2.

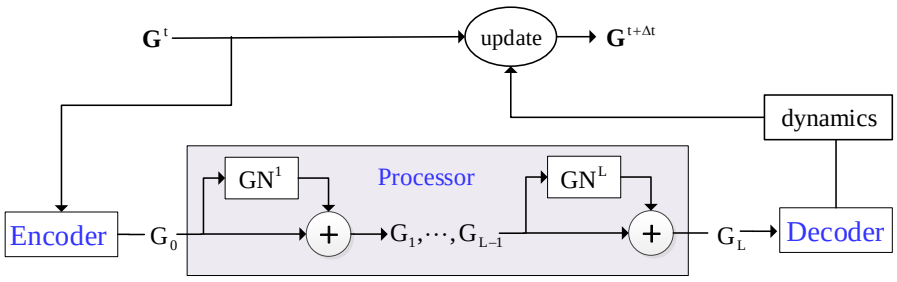


FIGURE 2. The network architecture of PIGNN.

1. ENCODER.

   Encoder converts the unstructured mesh at time step t into a latent graph $G_0^t = \text{ENCODER}\big(\mathrm{u}(\mathbf{x}, t)\big)$. To achieve spatial equivalence, mesh nodes become graph nodes $V = \{\mathbf{x}_i\}_{i=1}^N$, and mesh edges become bidirectional edges $E^G = \{\boldsymbol{e}_{ij}\}$ of the graph. As mentioned above, each node and edge is associated with a node feature $\boldsymbol{v}_i$ and an edge feature $\boldsymbol{e}_{ij}$, and their settings are the same as before. The node/edge features are concatenated as input

node/edge features, which are then transformed into high-level features using several multilayer perceptrons (MLPs) to obtain sufficient expressive power.

2. PROCESSOR.

At time step t, the processor consists of a stack of message passing blocks named GN (Graph Network) blocks, which models interactions among nodes and propagates information by running L steps to generate a sequence of updated latent graph $\boldsymbol{G}^t = (G_1^t, \cdots, G_L^t)$, where each block $l \in \{1, \cdots, L\}$ is computed based on the output of the previous block, i.e., $G_l^t = GN(G_{l-1}^t)$. In a GN block, computations proceed from edges to nodes, which contains two update functions and an aggregation function, following three procedures: updating the edge feature for each edge, then aggregating the updated edge features and projecting them to node features, and finally updating the node feature using the aggregated edges information. Mathematically,

$$\text{updating edge feature :} \quad \tilde{\boldsymbol{e}}_{ij,(l)} = \phi^e\big(\boldsymbol{e}_{ij,(l)}, \boldsymbol{v}_{i,(l)}, \boldsymbol{v}_{j,(l)}\big), \tag{4}$$

$$\text{aggregating edge features per node :} \quad \bar{\boldsymbol{e}}_{i,(l)} = \bigoplus_{j \in \mathcal{N}(i)} \tilde{\boldsymbol{e}}_{ij,(l)}, \tag{5}$$

$$\text{updating node feature :} \quad \tilde{\boldsymbol{v}}_{i,(l)} = \phi^v\big(\bar{\boldsymbol{e}}_{i,(l)}, \boldsymbol{v}_{i,(l)}\big), \tag{6}$$

where ϕ^e and ϕ^v are edge and node update functions respectively. These functions are mapped across all edges/nodes to compute per-edge/per-node update. In our implementation, MLPs are chosen as internal edge and node update functions because of its universal approximation property. The subscript (l) represents the block index. $\bigoplus$ denotes a permutation invariant aggregate function (such as sum, mean, maximum), which takes a set of edge updates as input and outputs a single element by projecting it onto the i-th node to represent the aggregated information. $\mathcal{N}(i)$ denotes indices of neighbours referring to a set of adjacent nodes that are connected to node i via edges. G_0^t is provided as the input of the processor's first GN block, we initialize the node features of graph G_0^t to the current PDE solutions, i.e., $\boldsymbol{v}_{i,(0)} = \mathbf{u}(\mathbf{x}_i, t)$. Based on the above illustration, we see that the processor computes interactions among related nodes and edges, propagates the information via message passing steps and finally returns a well updated graph G_L^t, that is, $G_L^t = PROCESSOR\big(G_0^t\big)$.

3. DECODER.

Decoder extracts dynamics from the node features $\boldsymbol{v}_{i,(L)}$ of the final latent graph G_L^t, namely, $\boldsymbol{h}_{i,(L)} = \text{DECODER}\big(\boldsymbol{v}_{i,(L)}\big)$. As before, we use a MLP as the learned function for the decoder. To compute the next time step solutions, we integrate the output $\boldsymbol{h}_{i,(L)}$ using a forward-Euler integrator with time interval $\Delta t = 1$: $\mathbf{u}(\mathbf{x}_i, t + \Delta t) = \mathbf{u}(\mathbf{x}_i, t) + \boldsymbol{h}_{i,(L)}$. Therefore, decoder's output represents updated quantities of the PDE system for two consecutive time steps, which is semantically meaningful to the update process.

2.3. **Physics-informed loss function.**

To capture generalizable dynamic patterns consistent with underlying physical laws, one of the common techniques is incorporating physical constraints into the loss function of the model. The basic concept is to tune the model trainable parameters and unknown PDE coefficients so that the network can fit the observations

while satisfying constraints defined by the underlying PDEs. In order to propose a well-posed optimization problem to improve solution accuracy and facilitate convergence, BCs/ICs are strictly enforced during network training. That is, IC is assigned to the nodes of the graph at the initial time step, and BC is assigned to the predicted solutions before computing PDE loss. The motivation for hard embedding BCs/ICs is to strictly incorporate all known information into the network to guide the network to quickly output physical plausible predictions.

Therefore, for the forward problem, our physics-informed loss function is simply the residual physics loss, which is given by

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}_p) = \mathcal{L}_{PDE}(\boldsymbol{\theta}; \mathcal{D}_p), \tag{7}$$

where $\boldsymbol{\theta}$ represents the trainable parameters of the model. $\mathcal{D}_p = \{\mathbf{x}_i^p, t_p\}_{i=1}^{N_p}$ is collocation points set for $p = 1, \cdots, N_{t_p}$.

For the inverse problem, the unknown parameter $\boldsymbol{\lambda}$ can be inferred by assimilating the observation data in a soft manner, where the above physics-informed loss is augmented by a data loss. That is, the following loss function is formulated:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathcal{D}_p, \mathcal{D}_d) = (1-\gamma)\mathcal{L}_{PDE}(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathcal{D}_p) + \gamma \mathcal{L}_{Data}(\boldsymbol{\theta}; \mathcal{D}_d), \tag{8}$$

where γ is a weighted parameter of the data loss, $\mathcal{D}_d = \{\mathbf{y}(\mathbf{x}_i^d, t_d)\}_{i=1}^{N_d}$ is the observation data set. It is worth noting that the observed time point $t_d\,(d = 1, \cdots, N_{t_d})$ may not equal the predicted time point t_p.

In the discrete-time setting, $\{\mathbf{x}_i^p, t_p\}$ selected from the interior mesh nodes and the corresponding time points acts as collocation points for calculating the PDE loss term $\mathcal{L}_{PDE}(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathcal{D}_p)$. Additional data loss will be assessed when the inverse problem is considered. The observation data $\{\mathbf{y}(\mathbf{x}_i^d, t_d)\}$ and the collocation points will be superimposed together to calculate the data loss and the PDE loss. N_p and N_d are the number of collocation points and observations for network training, respectively.

Concretely, the PDE loss aims to ensure consistency with physical laws, which is defined as the mean of squared residuals given by

$$\mathcal{L}_{PDE}(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathcal{D}_p) = \mathbb{E}\big[\big\|\mathcal{R}(\mathbf{x}^p, t_p; \boldsymbol{\theta}, \boldsymbol{\lambda})\big\|_2^2\big],$$

where $\|\cdot\|_2$ represents the l_2 norm, and the PDE residual is

$$\mathcal{R}(\mathbf{x}^p, t_p; \boldsymbol{\theta}, \boldsymbol{\lambda}) = \frac{\partial \mathbf{u}(\mathbf{x}^p, t_p; \boldsymbol{\theta})}{\partial t} + \mathcal{F}\big[\mathbf{u}, \nabla_{\mathbf{x}}\mathbf{u}, \Delta_{\mathbf{x}}\mathbf{u}, \cdots; \boldsymbol{\lambda}\big] - \mathbf{f}(\mathbf{x}^p, t_p).$$

The data loss measures the MSE between observations $\mathbf{y}(\mathbf{x}^d, t_d)$ and predictions $\mathbf{u}_{pred}(\mathbf{x}^d, t_d; \boldsymbol{\theta})$ of PIGNN,

$$\mathcal{L}_{Data}(\boldsymbol{\theta}; \mathcal{D}_d) = \mathbb{E}\big[\big\|\mathbf{y}(\mathbf{x}^d, t_d) - \mathbf{u}_{pred}(\mathbf{x}^d, t_d; \boldsymbol{\theta})\big\|_2^2\big].$$

With the above spatiotemporal discretization scheme, $\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\lambda})$ can be equivalently expressed as,

$$\begin{aligned} \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathcal{D}_p, \mathcal{D}_d) \quad = \quad & \frac{1-\gamma}{N_p N_{t_p}} \sum_{p=1}^{N_{t_p}} \sum_{i=1}^{N_p} \big\|\mathcal{R}(\mathbf{x}_i^p, t_p; \boldsymbol{\theta}, \boldsymbol{\lambda})\big\|_2^2 \\ & + \frac{\gamma}{N_d N_{t_d}} \sum_{d=1}^{N_{t_d}} \sum_{i=1}^{N_d} \big\|\mathbf{y}(\mathbf{x}_i^d, t_d) - \mathbf{u}_{pred}(\mathbf{x}_i^d, t_d; \boldsymbol{\theta})\big\|_2^2. \end{aligned} \tag{9}$$

At time step t, the predicted solutions of the next time step $t+\Delta t$ are separated from the calculation graph and then flowed into the network to calculate solutions for the step after next $t+2\Delta t$. The network repeats this procedure to calculate predictions for all training time steps, and accumulates the losses of all steps to update the network parameters in one go. Repeat the above process until a specified number of training iterations is reached or the network converges. A well optimized PIGNN model with optimal parameters $\boldsymbol{\theta}^*$ can not only approximate the PDE solutions, but also identify the unknown parameter of PDEs $\boldsymbol{\lambda}^*$, namely,

$$\{\boldsymbol{\theta}^*, \boldsymbol{\lambda}^*\} = \arg\min_{\{\boldsymbol{\theta},\boldsymbol{\lambda}\}} \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathcal{D}_p, \mathcal{D}_d).$$

2.4. **Rules of selecting features.**

According to the form of Eq. (1), if a system has no additional source input, i.e., $\mathbf{f}(\mathbf{x}, t)$ equals 0, then node features contain the learned node solution at time step t : $\mathbf{u}(\mathbf{x}, t)$, and a node type vector indicating whether it is an interior node or a boundary node. While edge features consist of the relative displacement of two adjacent nodes i and j and its Euclidean distance. Namely,

node features: $\boldsymbol{v_i} = \{\mathbf{u}(\mathbf{x}, t)$, node_type$\}$,

edge features: $\boldsymbol{e_{ij}} = \{\mathbf{x}_{ij}, \|\mathbf{x}_{ij}\|\}$.

However, if the source term $\mathbf{f}(\mathbf{x}, t)$ is nonzero, it does affect the evolution of the system, so we add the source term that does not contain solution functions to node features to capture its variations in both spatial and time space. Researchers know the influence and importance of the external source on the system. We need to embed such expert knowledge into our model to enable the network to learn these key dynamic patterns, thereby enhancing the generalizability of the model. The edge features remain the same as above. In this case, the features of nodes and edges are as follows:

node features: $\boldsymbol{v_i} = \{\mathbf{u}(\mathbf{x}, t)$, node_type, $\mathbf{f}(\mathbf{x}, \mathrm{t})\}$,

edge features: $\boldsymbol{e_{ij}} = \{\mathbf{x}_{ij}, \|\mathbf{x}_{ij}\|\}$.

3. **Some computational issues on the graph.**

In this section, we will discuss some computational problems on the graph. Specifically, we discuss methods for computing discrete differential operators on the graph, and the interpolation method adopted in the inverse problem.

3.1. **Differential operators on the graph.**

We use finite difference method to compute discrete differential operators (time and spatial derivatives) on a graph.

3.1.1. *Time derivative.*

Recall that PIGNN iteratively generates solutions of the next time step $t+\Delta t$ according to solutions of the current time step t. By utilizing solutions of two consecutive time steps, we can estimate the first order time derivative term in Eq. (1). Under the Euler setting, either explicit, implicit or semi-implicit methods can be used to calculate the time derivative. The explicit method usually contains one or several forward computation steps. Implicit or semi-implicit method, such as Crank-Nicolson's scheme, requires a fixed point iterative solver. Numerical results

show that, compared to explicit methods, implicit or semi-implicit methods provide some stability for the outputs in admissible time steps, albeit at the expense of increased computation cost. In this paper, we choose the implicit update rule to calculate the time derivative.

To sum up, due to its simplicity and stability, we use the implicit/backward Euler difference quotient to approximate the time-derivative, and the implicit/backward Euler discretization of Eq. (1) is,

$$\frac{\mathbf{u}(\mathbf{x},t+\Delta t)-\mathbf{u}(\mathbf{x},t)}{\Delta t}+\mathcal{F}\big[\mathbf{u}(\mathbf{x},t+\Delta t),\nabla_{\mathbf{x}}\mathbf{u}(\mathbf{x},t+\Delta t),\cdots;\boldsymbol{\lambda}\big]=\mathbf{f}(\mathbf{x},t+\Delta t). \quad (10)$$

3.1.2. *Gradient operator.*

In PINN, spatial derivatives in the PDE loss are calculated by the automatic differentiation technique, which utilizes the accumulated values during code execution and the chain rule of differential calculus to compute derivatives. However, this technique is not applicable to our framework due to computational complexity in backpropagation. Our PIGNN proceeds in an iterative manner, and the gradient of the solution function at each node depends on historical results up to the initial time, which leads to memory resource pressure and computational complexity. On a regular grid, the gradient of solution $\mathbf{u}$ at node $\mathbf{x}_i$ is a vector whose components are the derivatives of $\mathbf{u}$ at $\mathbf{x}_i$ along the axes. Each partial derivative can be computed by stencils of backward/forward/central finite difference. However, on an irregular mesh, it is infeasible to directly calculate the derivative along the axis. On the basis of least square method, we propose a simple gradient estimation method. Using the first-order Taylor expansion, we can construct a linear system. An estimation of the gradient is then obtained by learning the best fitting solution (least squares solution) of the linear system.

Let $\mathbf{x}_i=(\mathrm{x}_i^1,\cdots,\mathrm{x}_i^d)^{\mathrm{T}}\in\mathbb{R}^d$, $\mathbf{u}(\mathbf{x})=\mathbf{u}(\mathbf{x},t)$ for any fixed t, $\mathcal{N}(\mathbf{x}_i)=\{\mathbf{x}_{j1},\cdots,\mathbf{x}_{jp}\}$ be the node set of one-ring neighbours of node $\mathbf{x}_i$, $p=|\mathcal{N}(\mathbf{x}_i)|$ be the number of neighbours of node $\mathbf{x}_i$.

According to the first-order Taylor expansion of solutions at node $\mathbf{x}_i$, we have

$$\mathbf{u}(\mathbf{x}_j)-\mathbf{u}(\mathbf{x}_i)=(\mathbf{x}_j-\mathbf{x}_i)^{\mathrm{T}}\nabla\mathbf{u}(\mathbf{x}_i),\ \forall j\in\mathcal{N}(\mathbf{x}_i). \quad (11)$$

Without loss of generality, we assume that the number of neighbours is larger than the dimension of the above linear system, the system is overdetermined and only admits a least squares solution. Utilizing the least squares technique, we obtain an estimation of the gradient by solving the linear system.

Let $\boldsymbol{A}_i$ be the $p\times d$ matrix obtained by collecting all edges $\mathbf{x}_j-\mathbf{x}_i$, $\boldsymbol{U}_i$ be the column vector consisting of differences of node solutions connecting the corresponding edges $\mathbf{u}(\mathbf{x}_j)-\mathbf{u}(\mathbf{x}_i)$, concretely,

$$\boldsymbol{A_i} = \begin{pmatrix} (\mathbf{x}_{j1}-\mathbf{x}_i)^{\mathrm{T}} \\ \vdots \\ (\mathbf{x}_{jp}-\mathbf{x}_i)^{\mathrm{T}} \end{pmatrix} = \begin{pmatrix} \mathrm{x}_{j1}^1-\mathrm{x}_i^1 & \cdots & \mathrm{x}_{j1}^d-\mathrm{x}_i^d \\ \vdots & \ddots & \vdots \\ \mathrm{x}_{jp}^1-\mathrm{x}_i^1 & \cdots & \mathrm{x}_{jp}^d-\mathrm{x}_i^d \end{pmatrix}_{p\times d},$$

$$\boldsymbol{U_i} = \begin{pmatrix} \mathbf{u}(\mathbf{x}_{j1})-\mathbf{u}(\mathbf{x}_i) \\ \vdots \\ \mathbf{u}(\mathbf{x}_{jp})-\mathbf{u}(\mathbf{x}_i) \end{pmatrix}_{p\times 1}.$$

Then the linear system Eq. (11) can be written as $\boldsymbol{U}_i = \boldsymbol{A}_i \nabla \mathbf{u}(\mathbf{x}_i)$, and its least square solutions, namely, the estimation of the gradient at node $\mathbf{x}_i$ is

$$\nabla \hat{\mathbf{u}}(\mathbf{x}_i) = \left(\boldsymbol{A}_i^{\mathrm{T}} \boldsymbol{A}_i\right)^{-1} \boldsymbol{A}_i^{\mathrm{T}} \boldsymbol{U}_i. \tag{12}$$

3.1.3. *Laplace operator.*

Another commonly used differential operator is the Laplace operator. It is worth noting that the following method we proposed is general, which can be used to calculate other high-order differential operators similarly. Commonly used methods to compute the Laplace operator on graphs (unstructured meshes) include Taubin's discretization [29, 30], Fujiwara's discretization [31], Desbrun et al.'s discretization [32], Mayer's discretization [33], and Meyer et al.'s discretization [34]. All these methods adopt a general form which can be expressed in our notations as:

$$\Delta \mathbf{u}(\mathbf{x}_i, t) = \sum_{j \in \mathcal{N}(\mathbf{x}_i)} \omega_{ij} \big[\mathbf{u}(\mathbf{x}_j, t) - \mathbf{u}(\mathbf{x}_i, t)\big], \tag{13}$$

where $\Delta = \operatorname{div}(\nabla) = \nabla \cdot \nabla$ denotes the Laplace operator, $\boldsymbol{\omega}_i = (\omega_{i1}, \cdots, \omega_{ip})^{\mathrm{T}}$ is the coefficient/weight vector to be determined.

Remark 1. We can interpret the Eq. (13) in the following way. Recalling that the Gauss Divergence Theorem says, the surface integral of a vector field $\boldsymbol{F}$ over a closed surface S is equal to the volume integral of the divergence of the vector field $\boldsymbol{F}$ over the volume V enclosed by the closed surface, i.e., $\iint_S \boldsymbol{F} \cdot \vec{\boldsymbol{n}} \, \mathrm{d}S = \iiint_V (\nabla \cdot \boldsymbol{F}) \mathrm{d}V$. By specializing the theorem to the plane, then the following special equation holds,

$$\int_{\partial S} \boldsymbol{F} \cdot \vec{\boldsymbol{n}} \, \mathrm{d}s = \iint_S (\nabla \cdot \boldsymbol{F}) \mathrm{d}S. \tag{14}$$

Letting $\boldsymbol{F} = \nabla \mathbf{u}$ and paying attention to the Eq. (11), we see that, taking the small area around node $\mathbf{x}_i$ as the domain, according to Eq. (14), the Laplace value of node $\mathbf{x}_i$ can alternatively be approximated by the sum of directional derivatives along the domain boundary, which is approximated by Eq. (13). □

Noting from the coefficient expressions of classical discretizations, such as combinatorial weights, mean-value coordinates weights, and cotan weights, we see that coefficients only related to the spatial locations of adjacent nodes, and have nothing to do with values of solutions. That is to say, for the PDE system, when the discretization scheme on the computational domain is fixed, the generated mesh is determined, and the coefficients $\boldsymbol{\omega}$ is also determined and remains unchanged. On this basis, we estimate the coefficients by selecting a set of test functions and applying the ordinary least squares technique.

In order to calculate the second order derivatives at node $\mathbf{x}_i$, we choose a set of basis functions whose degree don't exceed $m\,(m \geq 2)$ as test functions $\tilde{\mathbf{u}}^k$, $k = 1, \cdots, K$, where K is the number of test functions. These basis functions are terms (excluding the constant term) of a d-element polynomial of m-th degree, i.e., $\tilde{\mathbf{u}}^k(x, y, t) \in \Psi_d = \{x_1, \cdots, x_d, x_1^2, x_1 x_2, \cdots, x_d^2, \cdots, x_1^m, \cdots, x_d^m\}$.

Based on the selected test functions, by finding the least squares solution of Eq. (13), we obtain the following estimation of the coefficient vector,

$$\hat{\boldsymbol{\omega}}_i = \left(\boldsymbol{C}_i^{\mathrm{T}} \boldsymbol{C}_i\right)^{-1} \boldsymbol{C}_i^{\mathrm{T}} \boldsymbol{B}_i, \tag{15}$$

where,

$$\hat{\boldsymbol{\omega}}_i = [\hat{\omega}_{i1}, \cdots, \hat{\omega}_{ip}]^T, \qquad \tilde{\mathbf{u}}_{ji}^k = \tilde{\mathbf{u}}^k(\mathbf{x}_j, t) - \tilde{\mathbf{u}}^k(\mathbf{x}_i, t),$$

$$\boldsymbol{C}_i = \begin{pmatrix} \tilde{\mathbf{u}}_{1i}^1 & \cdots & \tilde{\mathbf{u}}_{pi}^1 \\ \vdots & \ddots & \vdots \\ \tilde{\mathbf{u}}_{1i}^K & \cdots & \tilde{\mathbf{u}}_{pi}^K \end{pmatrix}_{K\times p}, \quad \boldsymbol{B}_i = \begin{pmatrix} \Delta\tilde{\mathbf{u}}^1(\mathbf{x}_i, t) \\ \vdots \\ \Delta\tilde{\mathbf{u}}^K(\mathbf{x}_i, t) \end{pmatrix}_{K\times 1}.$$

Substituting (15) into (13) gives us the estimation of the Laplace operator at node $\mathbf{x}_i$.

Remark 2.

(1) To eliminate the influence of $\mathbf{x}_i$ on ω_{ij} and improve computational stability, we translate the coordinate system and compute $\mathbf{u}(\mathbf{x}_j - \mathbf{x}_i, t)$ instead of $\mathbf{u}(\mathbf{x}_j, t)$, then Eq. (13) becomes

$$\Delta\mathbf{u}(0,t) = \sum_{j\in\mathcal{N}(\mathbf{x}_i)} \omega_{ij}\big[\mathbf{u}(\mathbf{x}_j - \mathbf{x}_i, t) - \mathbf{u}(0,t)\big]. \tag{16}$$

(2) In general, in a d-dimensional space (i.e, d variables), we can calculate that the number of non-constant terms for a test function with no more than m-th degree is $K = C_{d+m}^m - 1$, then to make Eq.(13) have a solution, the degree of the test function needs to satisfy $C_{d+m}^m \geq p+1$, where p is the number of neighbours of the current node. □

Definition 3.1. [Order of approximation] Let $\mathbf{x}_0 \in \mathbb{R}^p$, $\mathbf{x}_1, \cdots, \mathbf{x}_p \in \mathcal{N}(\mathbf{x}_0, R) = \big\{\mathbf{x} | \|\mathbf{x} - \mathbf{x}_0\| \leq R\big\}$). Define linear operator $\boldsymbol{\omega}$ as follows,

$$\boldsymbol{\omega}\mathbf{u}(\mathbf{x}_0) = \sum_{k=1}^{p} \omega_k\big[\mathbf{u}(\mathbf{x}_k) - \mathbf{u}(\mathbf{x}_0)\big], \text{ for } \mathbf{u} \in C^2\big(\mathcal{N}(\mathbf{x}_0, R)\big). \tag{17}$$

If for all $\mathbf{u} \in C^2\big(\mathcal{N}(\mathbf{x}_0, R)\big)$, we have

$$\Delta\mathbf{u}(\mathbf{x}_0) - \sum_{k=1}^{p} \omega_k\big[\mathbf{u}\big(\mathbf{x}_0 + r(\mathbf{x}_k - \mathbf{x}_0)\big) - \mathbf{u}(\mathbf{x}_0)\big] = o(r^s), \text{ as } r \to 0. \tag{18}$$

Then we say $\boldsymbol{\omega}$ is an approximation of Laplacian Δ with order of magnitude s at $\mathbf{x}_0$.

Lemma 3.2. *Let $\mathbf{x}_0 = \mathbf{0}$, $\mathbf{x}_1, \cdots, \mathbf{x}_p \in \mathcal{N}(\mathbf{0})$, $d = 2$, $\Psi = \Psi_2 = \{x, y, x^2, xy, y^2, \cdots, x^m, \cdots, y^m\}$, $\varphi_1 = x$, $\varphi_2 = y, \cdots$, $|\Psi| = n$, $\boldsymbol{\omega}_r$ be the least square solution of the following linear system*

$$\mathbf{Y} = \Phi(r)\boldsymbol{\omega}, \tag{19}$$

where

$$\boldsymbol{\omega} = (\omega_1, \cdots, \omega_p)^{\mathrm{T}}, \qquad \mathbf{Y} = \big(\Delta\varphi_1(\mathbf{0}), \cdots, \Delta\varphi_n(\mathbf{0})\big)^{\mathrm{T}},$$

$$\boldsymbol{\Phi}(r) = \begin{pmatrix} \varphi_1(r\mathbf{x}_1) & \cdots & \varphi_1(r\mathbf{x}_p) \\ \vdots & \ddots & \vdots \\ \varphi_n(r\mathbf{x}_1) & \cdots & \varphi_n(r\mathbf{x}_p) \end{pmatrix}_{n\times p}, \quad \boldsymbol{\Phi} = \boldsymbol{\Phi}(1).$$

Let $\boldsymbol{\eta}_r = r^2\boldsymbol{\omega}_r$, then

1) $\boldsymbol{\eta}_r$ is bounded.

2) Let $\{\boldsymbol{\eta}_n\}_{n=1}^{\infty}$ be any convergent subsequence of $\boldsymbol{\eta}_r$, if $\boldsymbol{\eta} = \lim\limits_{n\to\infty} \boldsymbol{\eta}_n$, then $\boldsymbol{\eta}$ is an approximation of Laplacian Δ with the largest order.

Proof. See Appendix A. □

3.2. **Interpolation method for inverse problem.**

In this section, we describe the interpolation method adopted in the inverse problem. Recall that in the inverse problem, some PDE parameters are unknown, but a few observations are available. For a given time point t_d and spatial location $\mathbf{x}_d$, the observed value $\mathbf{y}(\mathbf{x}_d, t_d)$ is known. By minimizing the difference between observations and PIGNN-based predictions at these given spatiotemporal points, we can estimate the unknown parameters. Noting that t_d may not equal t_p, to calculate the data loss, we need to know the predicted values at any time step t and spatial location $\mathbf{x}$, i.e., $\mathbf{u}_{pred}(\mathbf{x}, t; \boldsymbol{\lambda}, \boldsymbol{\theta})$. For simplicity, in the following, we denote $\mathbf{u}_{pred}(\mathbf{x}, t) = \mathbf{u}_{pred}(\mathbf{x}, t; \boldsymbol{\lambda}, \boldsymbol{\theta})$.

Assume that $t_d \in [t_p, t_{p+1})$, $\mathcal{N}(\mathbf{x}_d) = \{\mathbf{x}_1, \cdots, \mathbf{x}_l\}$ be l nearest nodes around $\mathbf{x}_d$. According to the Taylor expansion, we have

$$\begin{aligned} \mathbf{u}(\mathbf{x}_m, t_n) \quad \approx \quad & \mathbf{u}(\mathbf{x}_d, t_d) + \frac{\partial \mathbf{u}(\mathbf{x}_d, t_d)}{\partial t}(t_n - t_d) + (\mathbf{x}_m - \mathbf{x}_d)^{\mathrm{T}} \nabla \mathbf{u}(\mathbf{x}_d, t_d), \\ & \mathbf{x}_m \in \mathcal{N}(\mathbf{x}_d),\ n = p, p+1. \end{aligned} \tag{20}$$

The estimation of the desired prediction and its derivatives can be obtained with the least square technique, which is given by

$$\hat{\boldsymbol{H}} = \left(\boldsymbol{Q}^{\mathrm{T}}\boldsymbol{Q}\right)^{-1}\boldsymbol{Q}^{\mathrm{T}}\boldsymbol{Z}, \tag{21}$$

where,

$$\hat{\boldsymbol{H}} = \begin{pmatrix} \mathbf{u}_{pred}(\mathbf{x}_d, t_d) \\ \frac{\partial \mathbf{u}_{pred}(\mathbf{x}_d, t_d)}{\partial t} \\ \nabla \mathbf{u}_{pred}(\mathbf{x}_d, t_d) \end{pmatrix}, \qquad \boldsymbol{Z} = \begin{pmatrix} \mathbf{u}(\mathbf{x}_1, t_p) \\ \vdots \\ \mathbf{u}(\mathbf{x}_l, t_p) \\ \mathbf{u}(\mathbf{x}_1, t_{p+1}) \\ \vdots \\ \mathbf{u}(\mathbf{x}_l, t_{p+1}) \end{pmatrix}_{2l \times 1},$$

$$\boldsymbol{Q} = \begin{pmatrix} 1 & t_p - t & (\mathbf{x}_1 - \mathbf{x})^{\mathrm{T}} \\ \vdots & \vdots & \vdots \\ 1 & t_p - t & (\mathbf{x}_l - \mathbf{x})^{\mathrm{T}} \\ 1 & t_{p+1} - t & (\mathbf{x}_1 - \mathbf{x})^{\mathrm{T}} \\ \vdots & \vdots & \vdots \\ 1 & t_{p+1} - t & (\mathbf{x}_l - \mathbf{x})^{\mathrm{T}} \end{pmatrix}_{2l \times (d+2)}.$$

From Eq. (21), we can obtain the predicted value $\mathbf{u}_{pred}(\mathbf{x}, t)$ at any spatial-temporal point $(\mathbf{x}, t)$, therefore, for the observation point $(\mathbf{x}_d, t_d)$, its prediction $\mathbf{u}_{pred}(\mathbf{x}_d, t_d)$ is known, and data loss can be calculated.

4. **Experiments.**

In this section, we conduct a series of experiments on a group of canonical PDEs in both forward and inverse settings to demonstrate the effectiveness of our proposed PIGNN. We aim to answer the following questions.

Q1. How accurate is the proposed PIGNN in solving PDEs compared with the baseline model?

Q2. How does PIGNN perform in time extrapolation? (i.e., model settings are consistent with that of training time steps)?

Q3. Does PIGNN generalize well to other conditions not seen during training, such as other computation domains, spatial resolutions and ICs?

To answer these questions, we choose three nonlinear PDE systems ranging from simple to complex: heat equation, Burgers equation and FitzHugh-Nagumo (FN) equation in 2-dimensional (2D) domains with known/unknown parameters, and take the vanilla PINN as the baseline model. Heat equation is probably one of the simplest PDEs to describe the behavior of diffusive systems. Burgers equation [36] is an important nonlinear parabolic PDE widely used to model fluid dynamics, which directly incorporates the interaction between non-linear convection processes and diffusive viscous processes. The coupled FN equation [37] is an example of a nonlinear reaction-diffusion system that is introduced for the nerve impulse propagation in nerve axons to describe activator-inhibitor neuron activities excited by external stimulus.

The effectiveness of PIGNN for other equations such as Euler, Navier-Stokes and Maxwell is under investigation and related results will be presented in our future studies. It is rather remarkable that though PINN learns the global solution directly from training points, and PIGNN is a time-step iterative method. PINN is chosen as the baseline model for comparison because we believe that both methods can learn propagation trends of physical systems over the training time steps.

We consider the initial-boundary value problem with Dirichelet BCs in the computational domains for all examples. To verify the accuracy of our model, we choose specific ICs and BCs for heat and Burgers equations such that they have analytical solutions. The concrete settings for PIGNN and baseline model are illustrated in Appendix B. And concrete expressions of PDE examples are shown in Appendix C.

4.1. **Evaluation metrics.**

To evaluate the accuracy of predicted solutions, at time step t, we use the absolute error between the predicted solution $\mathbf{u}_{pred}(\mathbf{x}, t; \boldsymbol{\theta})$ and the real solution $\mathbf{u}^*(\mathbf{x}, t)$, given by,

$$\text{Error} = |\mathbf{u}_{pred}(\mathbf{x}, t; \boldsymbol{\theta}) - \mathbf{u}^*(\mathbf{x}, t)|. \tag{22}$$

While for the entire testing period, we use the accumulative root-mean-square-error (aRMSE) which is given by

$$\text{aRMSE} = \sqrt{\frac{1}{NN_t}\sum_{k=1}^{N_t}\sum_{i=1}^{N}\|\mathbf{u}_{pred}(\mathbf{x}_i, t_k; \boldsymbol{\theta}) - \mathbf{u}^*(\mathbf{x}_i, t_k)\|_2^2}, \tag{23}$$

where N_t is the total number of time steps within $[0, t]$.

4.2. **Results.**

In this subsection, we present results for three PDEs in both forward and inverse settings. Specifically, we start by showing results of the forward problem with long time steps to illustrate the accuracy and time extrapolability of our PIGNN. Then, a series of results with varying PDE parameters, computational domains and spatial resolutions are presented to demonstrate the generalizability of PIGNN. Moreover, we present parallel results to demonstrate that PIGNN can scale well to distributed multi-GPU systems to handle large-scale problems. In addition, a simple result

verifies the ability of PIGNN to handle inverse problems. Finally, we conduct an ablation study to obtain a deeper understanding of our PIGNN. We display two experiments to elucidate the effects of the number of GN blocks and proper node features on model performance.

Our main finding is that PIGNN outperforms the PINN baseline for almost all cases, and in particular it generalizes better to more complex situations. The main advantage of our method is that models trained in small domains with simple settings possess fantastic fitting ability, and can be directly applied to more complex situations with large domains. This is extremely important in practice, as there is often a demand for rapid assessment or response. When the computational domain, ICs or BCs change, traditional numerical methods need to be recalculated, which is very time-consuming. DL-based methods with poor generalization ability may produce incorrect predictions. While our method can quickly give high confidence estimations.

4.2.1. *Extrapolability of the forward problem.*

We train an independent PIGNN/PINN model for each PDE example to obtain approximate solutions under the corresponding training settings. Specifically, for heat equation, we train the PIGNN model for 50 time steps with a time interval $\Delta t = 10^{-4}$, while for Burgers and FN equations, we only let the PIGNN model learn 10 time steps with $\Delta t = 10^{-3}$. We take the model with the least loss as the test model to evaluate the model's performance, hoping that it can infer the subsequent solutions. During testing, we predict hundreds or thousands of time steps for each example to demonstrate the time extrapolability of our PIGNN. For fair comparison, in PINN training the computation area and training time steps are the same as those in our approach. We randomly select a certain number of collocation points from both the computation domain and temporal scope $[0, \text{time steps} \times \Delta t]$. Since PINN is meshless, when testing, the inputs to the PINN model are mesh points' coordinates of the corresponding PIGNN model and time step. That is, PIGNN's mesh points are taken as the domain sampling points.

Figures 3-5 describe the forward solution snapshots for each example predicted by PIGNN and PINN, compared against the analytical solutions. It is worth noting that there is no analytical solution to the FN equation, and we take results of the numerical commercial software as the reference. At the same time, to show the accuracy for each method, we provide snapshots of the absolute error at the corresponding time steps. Besides, we present the accumulative error curves with respect to time in Fig. 6 to further depict the accuracy of different methods.

In Fig 3, we select three representative snapshots of heat equation at time steps 50, 1000 and 2500, respectively. It can be seen that our PIGNN shows excellent agreement with the analytical solution in both training and extrapolation phases, while PINN fails to match the analytical result at longer time steps. It is clear from the last two columns of the figure that the absolute error of PINN is much larger than that of PIGNN. This phenomenon is further verified by the left subplot of Fig. 6, where the aRMSE curve of PIGNN is consistently lower than that of PINN.

Figure 4 exhibits similar performance for the Burgers equation, where we present predictions at time steps 5, 500 and 1500. As can be seen from the right subplot of Fig. 6, the aRMSE of PIGNN for the Burgers equation starts out relatively larger than that of PINN, then the curve flattens out and drops below the PINN curve at half steps. Since we fail to train a PINN model for the FN equation, we only present

the extrapolation results of PIGNN at time steps 5 and 500 in Fig. 5, compared with the reference solution. Similar to the above two cases, the minor difference in training and extrapolation phases demonstrate the great learning potential of our PIGNN.

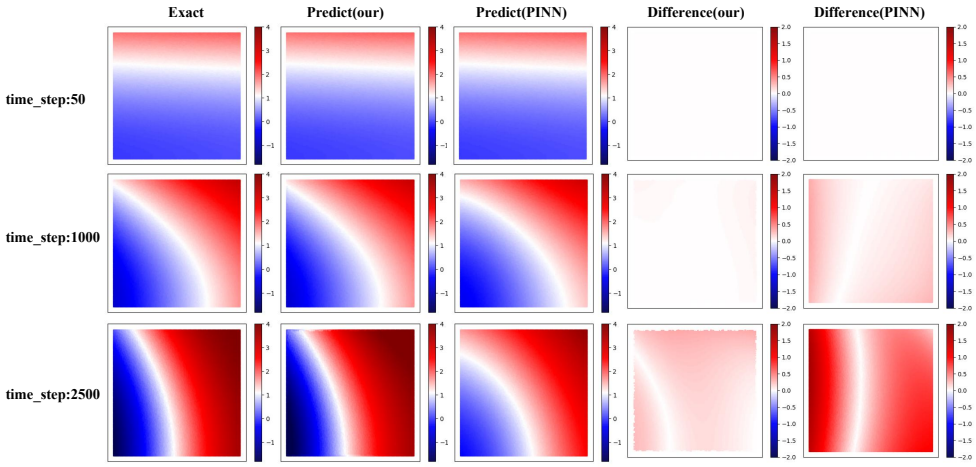


FIGURE 3. Comparison of time extrapolability between PIGNN and PINN for heat equation. Computational domain: $[0,1]\times[0,1]$, mesh density=100.

.

4.2.2. *Generalization on the forward problem.*

As aforementioned, fast estimations or responses are often required in practice. When computational domains, ICs or BCs change, numerical methods need to be recalculated, which is very time-consuming. DL-based methods with poor generalizability may produce incorrect estimations. Therefore, in this subsection, by carrying out a series of experiments, we demonstrate that our PIGNN generalizes well beyond training scenarios for varying PDE parameters, computational domains and mesh sizes (or spatial resolutions).

(1) Changing PDE parameters

Firstly, we change the PDE parameter in heat equation from $b=2$ to $b=8$. From Eq. (34) and the definition of BC, we known that different parameters actually mean different PDE source terms $f(t,x,y)$ and BCs. Fig. 7 shows a visual comparison between PIGNN, PINN and analytical solutions. It can be clearly seen that PINN is unable to produce correct results, while our PIGNN performs well even for long time steps.

(2) Changing spatial resolutions

Secondly, we maintain the same size of computational domain and increase/decrease the mesh size, namely, the spatial resolution is finer/coarser than the default settings. Since PINN is meshless, changing resolutions of the PINN model implies adopting another sampling strategy. Figures 8-9 show snapshots of solutions to heat and Burgers equations predicted by PINN and PIGNN under finer and coarser

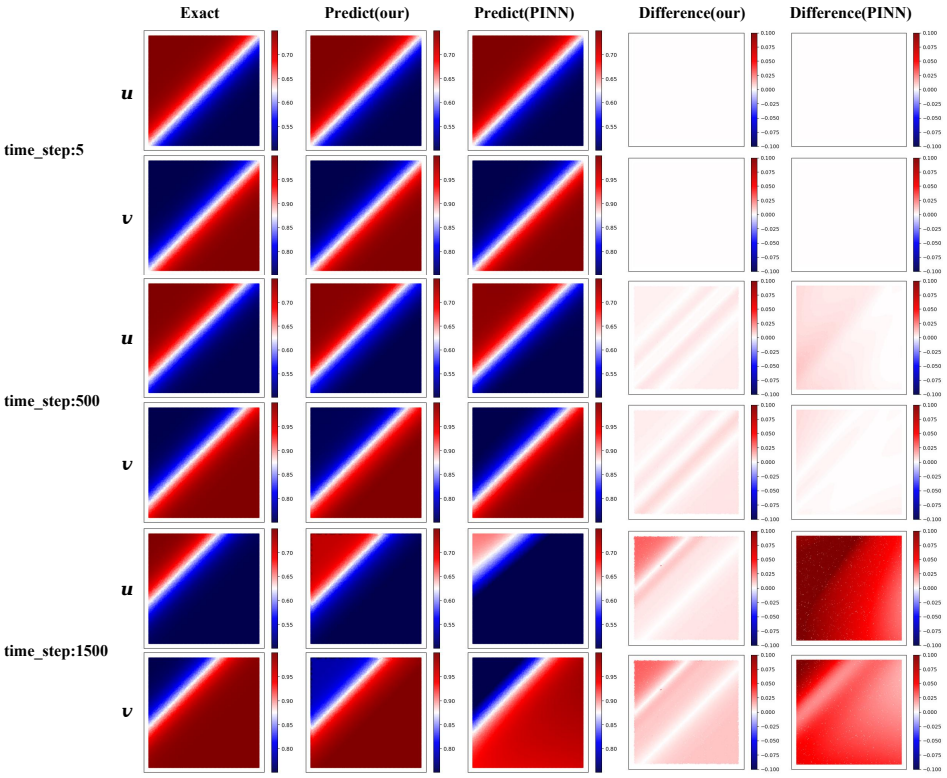


FIGURE 4. Comparison of time extrapolability between PIGNN and PINN for Burgers equation. Computational domain: $[0,1]\times[0,1]$, mesh density=100.

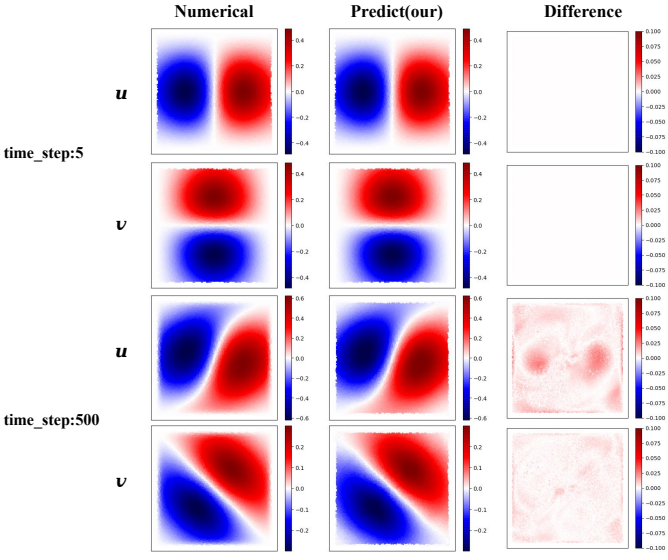


FIGURE 5. Time extrapolability of PIGNN compared with numerical results for FN equation. Computational domain: $[0,1]\times[0,1]$, mesh density=100.

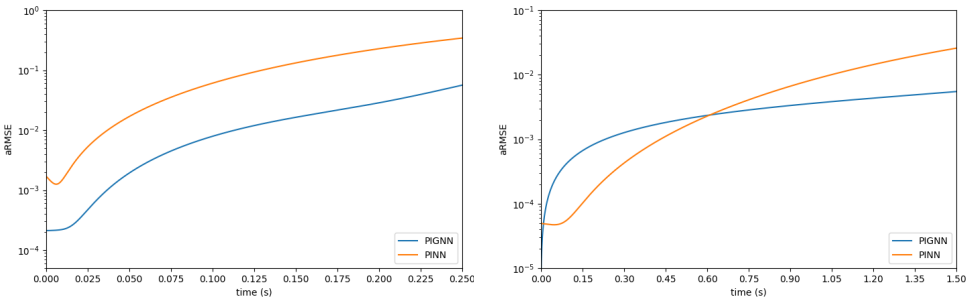


FIGURE 6. Comparison of aRMSE curves between PIGNN and PINN for heat (left) and Burgers (right) equations. Computational domain: $[0,1]\times[0,1]$, mesh density=100.

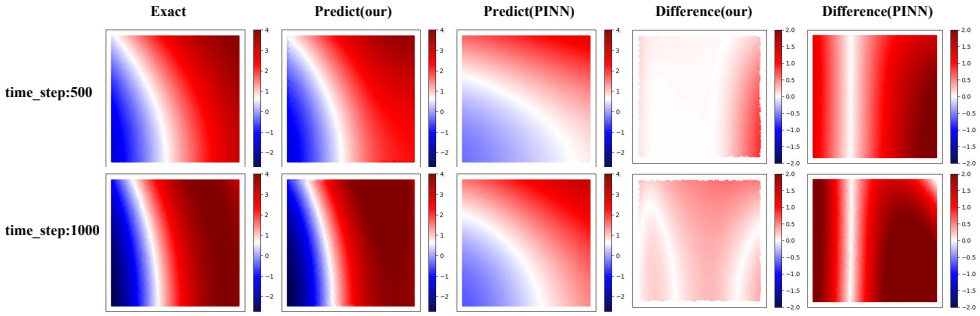
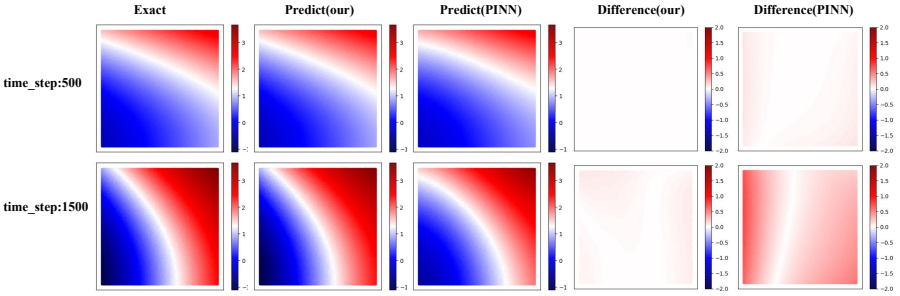


FIGURE 7. Generalization performance of the heat equation with parameter b changed from 2 to 8. Computational domain: $[0,1]\times[0,1]$, mesh density=100.

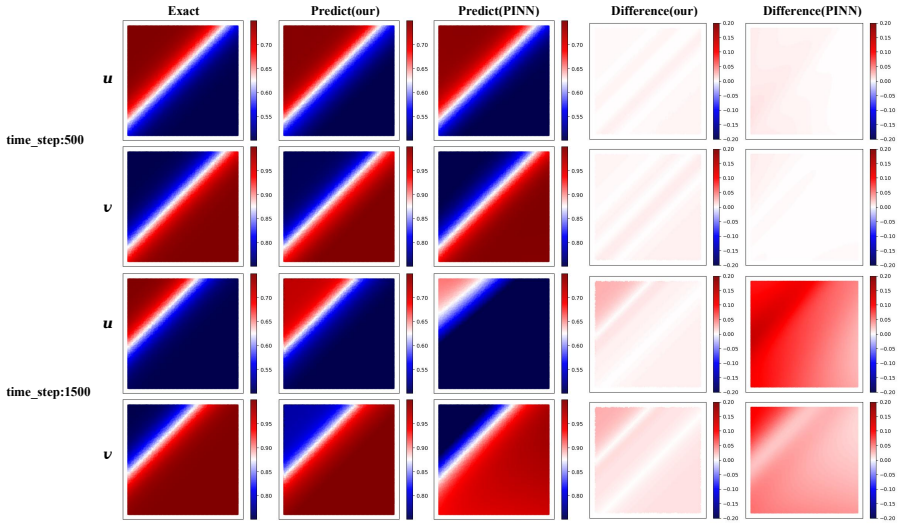
resolutions at time steps 500 and 1500, respectively. For the finer resolution case, we increase the mesh density from 100 to 200, while in the coarser case, we decrease the mesh density from 100 to 50. As can be seen from the last two columns of each figure, the absolute error of PINN is larger than that of PIGNN for both equations. This phenomenon illustrates that our PIGNN is much more robust to changes in spatial resolution.

(3) Enlarging computational domain

Next, we expand the computational domain by 4 times, keeping the same mesh size (spatial resolution), that is, the computational domain is enlarged from $[0,1]\times[0,1]$ to $[0,2]\times[0,2]$, and the mesh density is increased from 100 to 400. We predict solutions of heat and Burgers equations for 1000 and 1500 steps, and visualize the rollout performance in Fig. 10, respectively. The top subplot shows that for heat equation PINN is extremely sensitive to changes in computational domain and outputs completely wrong results, while predictions of PIGNN are consistent with the analytical reference. And from the bottom subplot of Fig. 10, we see that our PIGNN has smaller errors than PINN at all time steps for Burgers equation. This experiment demonstrates that our PIGNN is also robust to changes in computational domain.



(a) heat



(b) Burgers

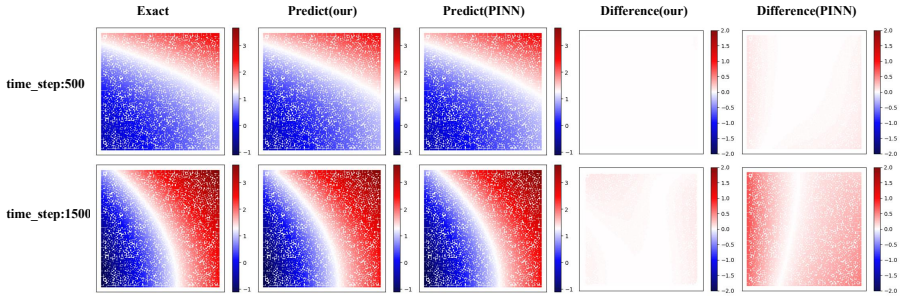
FIGURE 8. Generalization performances of heat (top) and Burgers (bottom) equations under the default computational domain with finer spatial resolution. Computational domain: $[0, 1] \times [0, 1]$, mesh density=200 (100 by default).

(4) Enlarged computational domain with coarser spatial resolution

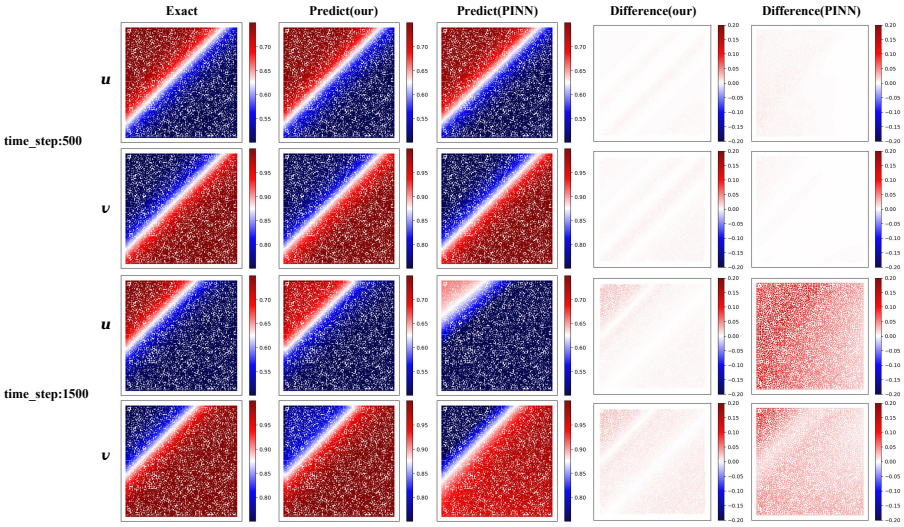
Furthermore, we expand the computational domain by 9 times, i.e., it is enlarged from $[0, 1] \times [0, 1]$ to $[0, 3] \times [0, 3]$, and let mesh density be 300 (100 by default). As a result, the spatial resolution becomes much coarser than default settings. Same as the previous experiment, we obtain 1000 and 1500 time steps predicted solutions of heat and burgers equations respectively and describe two time steps snapshots in Fig. 11. Similar to previous results, our PIGNN presents outstanding fitness to analytical solutions of both equations, while PINN can simulate part of dynamical patterns of the Burgers equation, but hardly in the heat equation.

(5) Enlarged computational domain with finer spatial resolution

Moreover, we expand the computational domain by 2 times, i.e., it is enlarged from $[0, 1] \times [0, 1]$ to $[0, 2] \times [0, 1]$, and let mesh density be 400 (100 by default). Namely, the spatial resolution is finer than default settings. Fig. 12 shows the visual



(a) heat

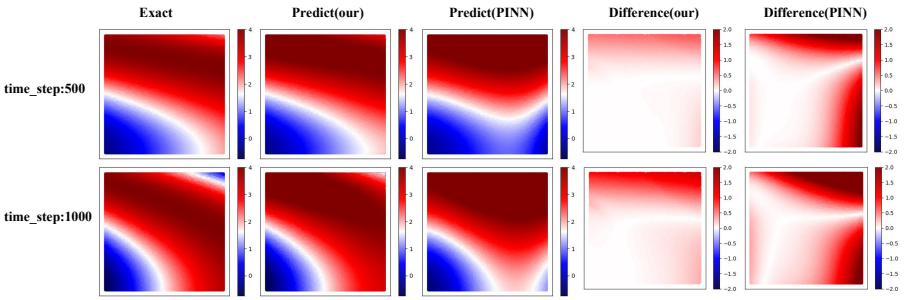


(b) Burgers

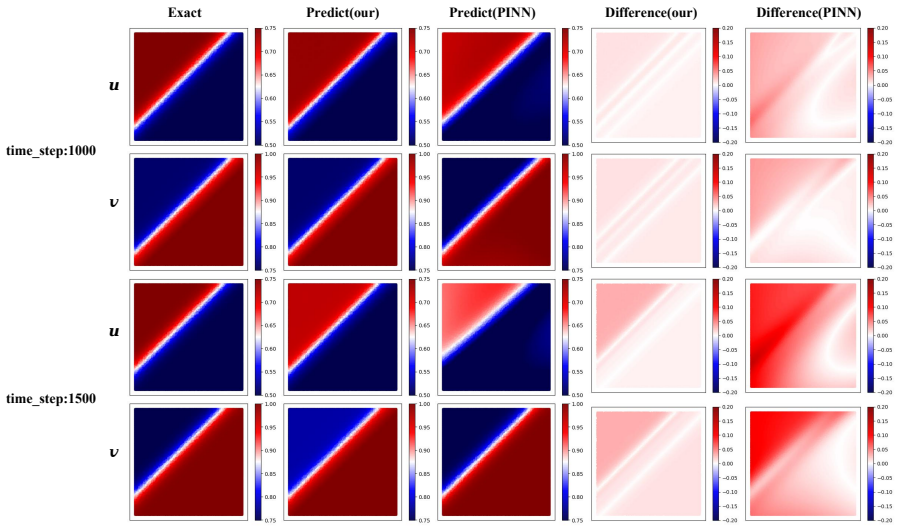
FIGURE 9. Generalization performances of heat (top) and Burgers (bottom) equations under the default computational domain with coarser spatial resolution. Computational domain: $[0,1] \times [0,1]$, mesh density=50 (100 by default).

predictions of PIGNN and PINN for heat and Burgers equations in comparison with the reference. As in previous three experiments, results in this case show similar patterns, and we don't go into details here.

In addition, in Fig. 13, we depict the aRMSE curves of heat (top) and Burgers (bottom) equations separately for all spatial resolution cases in enlarged computational domains. Let's first explain the meaning of the legend in the figure. 'initial' represents the case with default settings, 'remain' means the case in the enlarged computational domain ($[0,2] \times [0,2]$) with the same mesh size (density is 400), 'coarser' corresponds to the case in the enlarged computational domain ($[0,3] \times [0,3]$) with large mesh size (density is 300), and 'finer' refers to the case in the enlarged computational domain ($[0,2] \times [0,1]$) with small mesh size (density is 400). Dashed lines represent the error curves of PINN, while solid lines are curves of PIGNN. As can be seen from the above subgraph, for each case of the heat equation, the error curve of PIGNN is always lower than the corresponding PINN curve.



(a) heat

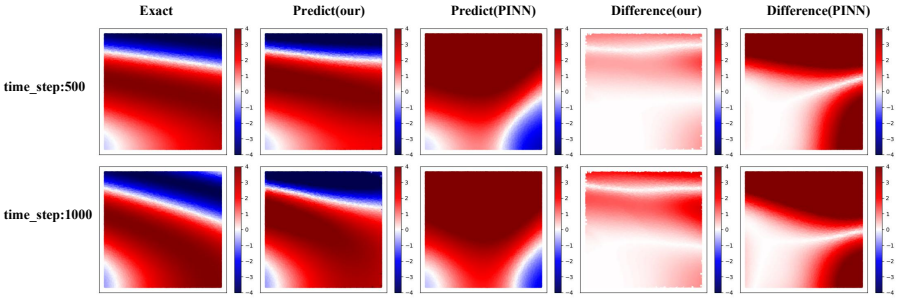


(b) Burgers

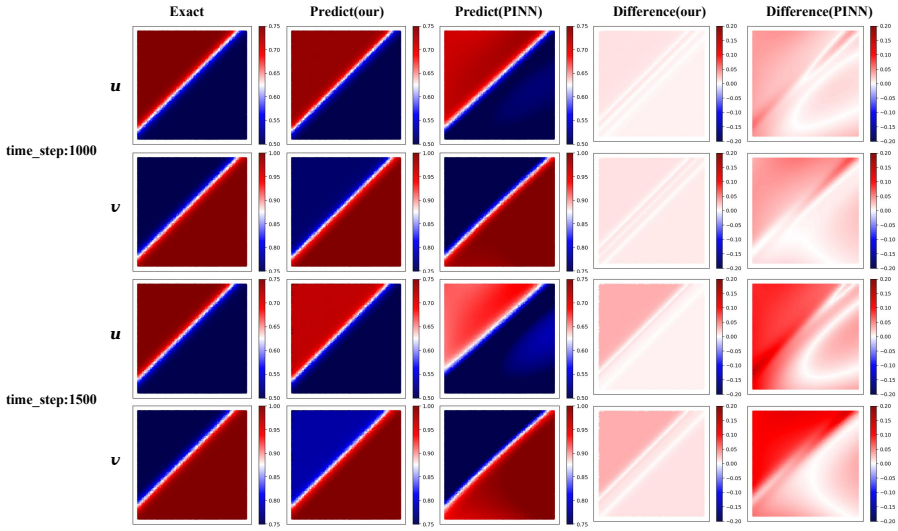
FIGURE 10. Generalization performances of heat (top) and Burgers (bottom) equations under the enlarged computational domain with the same spatial resolution. Computational domain: $[0, 2] \times [0, 2]$, mesh density=400. Default settings: computational domain: $[0, 1] \times [0, 1]$, mesh density=100.

Comparing all the PIGNN error curves, the default case has the smallest error, while the 'coarser' case possesses the largest error. However, from the subgraph below we see that for the Burgers equation, the phenomenon is a little different. For all PIGNN cases, the error curves do not vary too much and are concentrated around the default error curve. While for PINN cases, the default error curve is lower than that of PIGNN at first and then increases dramatically. After half of the time step, all PINN error curves increase rapidly, but PIGNN's error curves tend to be flat steadily. The results in this figure further validate the robustness of PIGNN to changes in computational domain and spatial resolution.

(6) Changing computational domain, spatial resolution and PDE parameters simultaneously



(a) heat



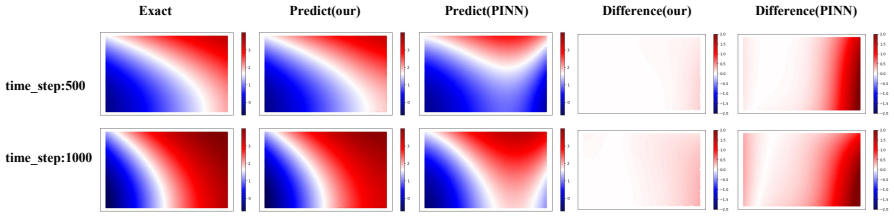
(b) Burgers

FIGURE 11. Generalization performances of heat (top) and Burgers (bottom) equations under the enlarged computational domain with coarser spatial resolution. Computational domain: $[0,3]\times[0,3]$, mesh density=300. Default settings: computational domain: $[0,1]\times[0,1]$, mesh density=100.

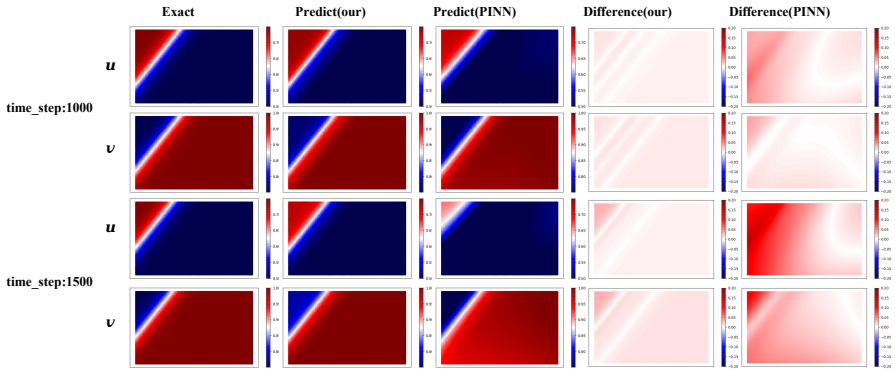
In the last part of this subsection, we expand the computational domain by 2 times with finer spatial resolution for heat and Burgers equations. That is, the computational domain is enlarged from $[0,1]\times[0,1]$ to $[0,1]\times[0,2]$ and the mesh density is increased from 100 to 300. We set $b=6$ (2 by default) and $R_e=60$ (80 by default) in the heat and Burgers equations, respectively. For the FN equation, to further demonstrate the fitting ability of PIGNN under complex settings in large domains, we train the PIGNN model with four groups of parameters at the default settings, and then change the IC with the following complex expression when testing

$$\begin{aligned} u &= \sin[200\pi(x+y)+0.9]+1.5\sin[200\pi xy(x-y)]+1.4\cos(\pi y),\\ v &= 0.9\sin[200\pi xy(x-0.5)(x-y)+0.9]+1.1\sin[50\pi(x-0.5)y+1.4]. \end{aligned}$$

Meantime, we expand the computational domain by 4 times, $[-1,1]\times[-1,1]$ and increase the mesh density to 1000.



(a) heat



(b) Burgers

FIGURE 12. Generalization performances of heat (top) and Burgers (bottom) equations under the enlarged computational domain with finer spatial resolution. Computational domain: $[0, 2] \times [0, 1]$, mesh density=400. Default settings: computational domain: $[0, 1] \times [0, 1]$, mesh density=100.

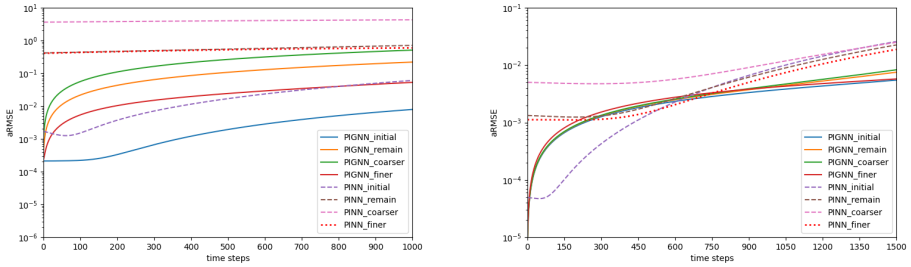


FIGURE 13. Comparison of accumulative error curves for heat (left) and Burgers (right) equations under different computational domains with different spatial resolutions.

All predictions are pictured in Figs. 14 and 15. It appears from Fig. 14 that, even when we change the PDE parameter, computational domain and spatial resolution simultaneously, our PIGNN still gives satisfactory predictions for both heat and Burgers equations. PINN does not perform well in the heat equation, but produces relatively better results in the Burgers equation. Frankly speaking, the error in the current case is larger than the error when changing the computational domain,

spatial resolution or PDE parameter separately. Fig. 15 shows that PIGNN is able to infer the high-frequency dynamic patterns never seen during training, suggesting that our model has the potential to generalize to more complex situations.

We think the above prominent strength comes from the architecture design of our graph network and appropriate choices of node features, which is conductive to learning local dynamic patterns of systems. Furthermore, by forcing the network to make next-step prediction on irregular meshes with distance-relative edge features, our model is encouraged to learn resolution-independent physics. In order to clearly show the results of FN equation with high-frequency dynamics, we provide an amplified version of result $\boldsymbol{u}$ at time step 80. It should be remarked here that since the CPU resources are already exhausted when the numerical software is working with a mesh density of 1000, we have no reference results in this case. This experiment also demonstrates the availability and substitutability of our PIGNN compared with numerical softwares, that is, our model is an alternative tool when the numerical software cannot perform calculations due to the large amount of computations on the same computer configuration.

4.2.3. *Scalability on the forward problem.*

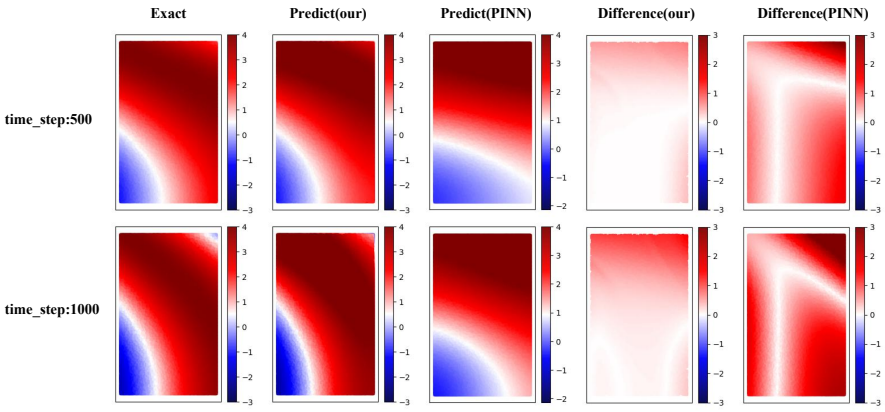
In this subsection, taking the Burgers equation as an example, we present parallel results of a large-scale PIGNN for distributed multi-GPU systems to demonstrate the scalability of our PIGNN. Since the focus of this paper is not the distributed parallelism of large-scale graph neural networks, technical details of parallel computing will not be introduced here, and we only show the results. From an implementation point of view, the parallelism of the large-scale PIGNN is simpler than other parallel algorithms because the model structure remains the same. Only the graph partition, the exchange and synchronization of boundary information of each subgraph need to be considered.

In this experiment, the computational domain is $[0, 28] \times [0, 28]$, and the mesh density is 2800, resulting in a total number of nodes of 12170332. The graph is divided into four subgraphs so that the number of nodes in each subgraph is as close as possible. Fig. 16 shows snapshots of solutions and differences for the large-scale problem in the training and extrapolation phase at time steps 5 and 1500. In order to clearly show the results of such high resolution, we provide an amplified version of result $\boldsymbol{v}$ at time step 5 in Fig. 17. As can be seen from Fig. 16 that the error is small even at very long time steps. This experiment verifies that our PIGNN can be well scaled to distributed systems to deal with large-scale problems.

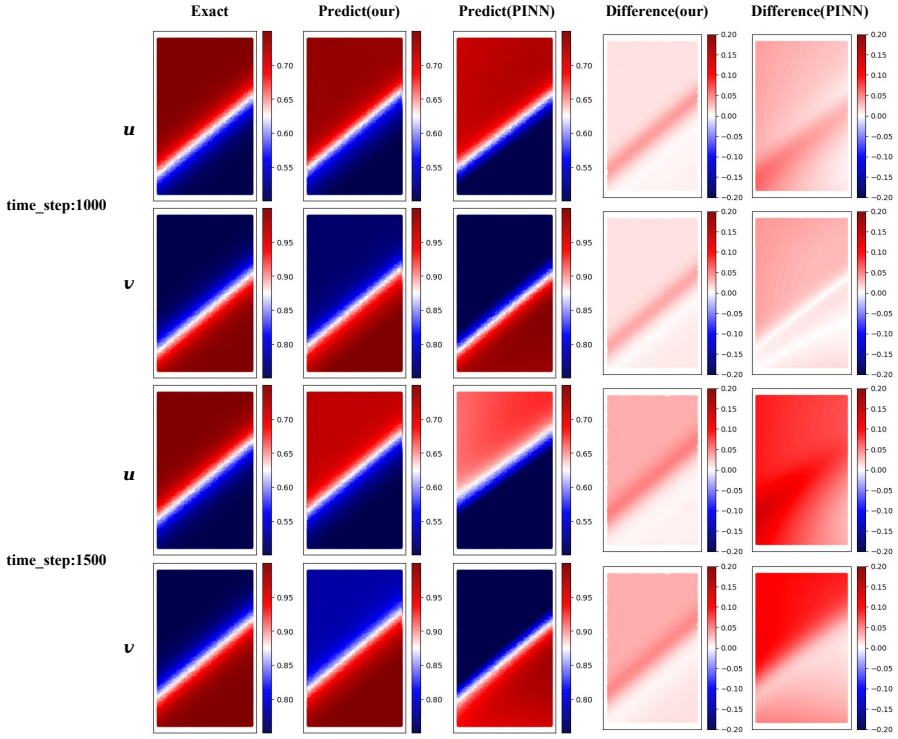
4.2.4. *Inverse problem.*

For the inverse problem, we take a simple heat equation as an example for explanation. We choose the following equation,

$$\begin{aligned} \frac{\partial u(x,y,t)}{\partial t} &= \Delta u(x,y,t) + f(x,y,t), \\ f(x,y,t) &= -kx\sin[ktx+y] + \cos(ktx+y)(k^2t^2+1), \\ (x,y) \in \Omega &= [0,1]\times[0,1],\ t>0, \end{aligned} \tag{24}$$



(a) heat



(b) Burgers

FIGURE 14. Generalization performance of heat (above) and Burgers (below) equations with different computational domain, spatial resolution and PDE parameter. heat: $b = 6$, Burgers: $R_e = 60$. Computation domain: $[0, 1] \times [0, 2]$, mesh density=300. Default settings: computational domain: $[0, 1] \times [0, 1]$, mesh density=100, $b = 2$, $R_e = 80$.

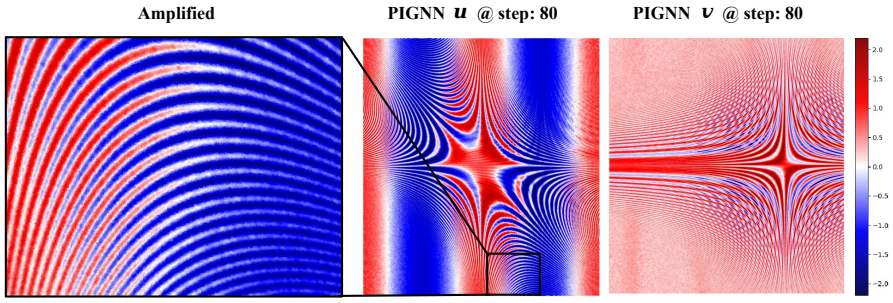


FIGURE 15. Generalization performance of the FN equation with different computational domain, spatial resolution and IC. Computation domain: $[-1,1]\times[-1,1]$, mesh density=1000. Default settings: computational domain: $[0,1]\times[0,1]$, mesh density=100.

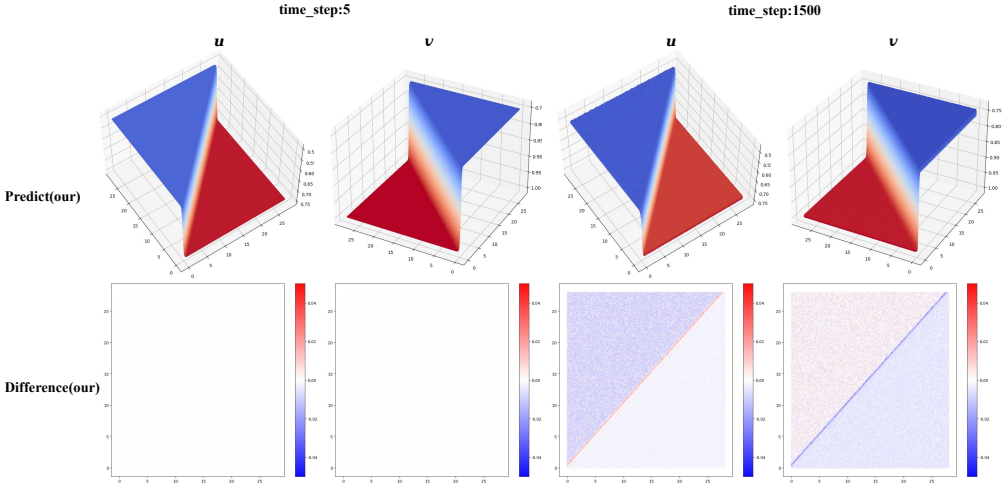


FIGURE 16. Generalization performance of large-scale PIGNN for Burgers equation. Computation domain: $[0,28]\times[0,28]$, mesh density=2800. Default settings: computational domain: $[0,1]\times[0,1]$, mesh density=100.

subject to the initial and boundary conditions

$$\begin{aligned} u(x,y,0) &= \cos(y), && (x,y)\in\Omega, \\ u(x,y,t) &= \cos(4tx+y), && (x,y)\in\partial\Omega,\ t>0. \end{aligned}$$

The analytical solution is

$$u(x,y,t)=\cos(ktx+y). \tag{25}$$

To highlight the ability of PIGNN of learning from scarce training data, we randomly sample 100 points from the computational domain, accounting for only 0.67% of total nodes, and select the corresponding analytical solutions as observations to form the training dataset. Fig. 18 shows the training process of our PIGNN under three different initial values of the unknown parameter k. It is clear from the figure that for arbitrary initial value, all curves converge to the same value as the model

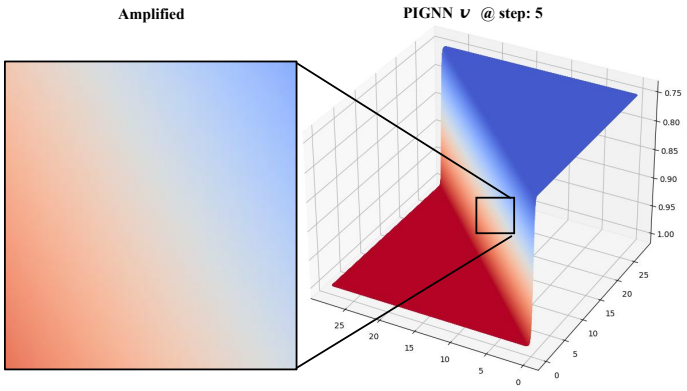


FIGURE 17. The amplified result $\boldsymbol{v}$ of large-scale PIGNN at time step 5 for the Burgers equation.

training process progresses, which implies that our PIGNN can identify the true parameter.

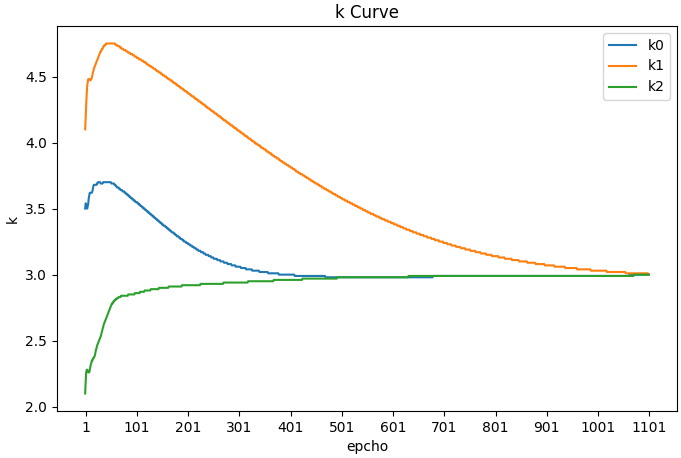


FIGURE 18. Training process of PIGNN under different initial values of unknown parameter of heat equation.

4.2.5. *Ablation study.*

In this subsection, two additional experiments are carried out to demonstrate the influences of the number of GN blocks and the choice of node features on model's performance.

(1) Effect of the number of GN blocks

In this experiment, we choose three sets of heat equation parameters $b = -2, 0.9, 8$, and test the performance of our PIGNN under one and two GN blocks, respectively. Fig. 19 describes the aRMSE curve with respect to time steps for each case. It can

be seen that for the same parameter value, the error curve of one GN block is consistently lower than that of two GN blocks, but the gap is quite small. We think this is due to the fact that each node is only sensitive to changes of its one-ring neighbours, and model with one GN block is sufficient enough to learn such dynamic patterns.

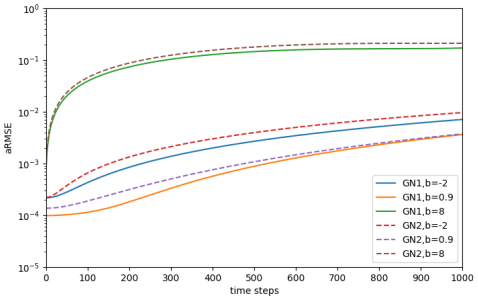


FIGURE 19. Generalization performance of heat equation with different number of GN blocks and PDE parameters.

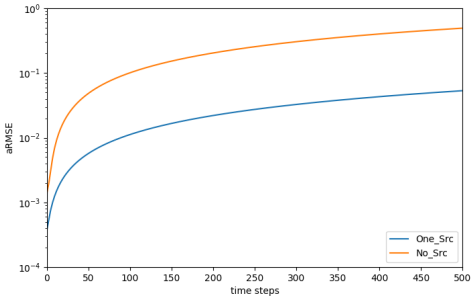


FIGURE 20. Comparison of aRMSE curves of heat equation with different choices of node features.

(2) Importance of appropriate node features

Here, we are going to demonstrate the importance of choosing appropriate node features. As we explained in subsection 2.4, if the source term of equation is nonzero, we have to add it to node features to capture the spatial dynamic pattern. We test this selecting rule by choosing two sets of node features. The first case refers to the default setting, that is, node features contain node solutions and node type vectors. In the second case, besides the default setting, the source term $f(t, x, y)$ in equation (34) is added to node features. We term these two cases: No_src and One_src. The aRMSE curves for both cases are presented in Fig. 20. Furthermore, we provide two representative snapshots in Fig. 21 to visualize the impact of different node features on PIGNN's predictions. Figure 20 clearly shows that the error of No_src is higher than that of the other case. The results of Figure 21 further confirm this conclusion, where the results of No_src show that the model without additional source feature rarely learns the dynamics of the system, while the model with One_src has successfully learned.

In the final part of this article, we make an overall comparison between our PIGNN and PINN from 10 aspects to further demonstrate the outstanding properties of our PIGNN. PIGNN can accurately model various PDE systems with good time extrapolability and can be well generalized to different computational domains, spatial resolutions (mesh size) and PDE parameters simultaneously. For details, please see Table 1.

5. Conclusion.

In this paper, we propose a novel PIGNN framework for solving both the forward and inverse problems governed by spatiotemporal PDEs. Thanks to the integration

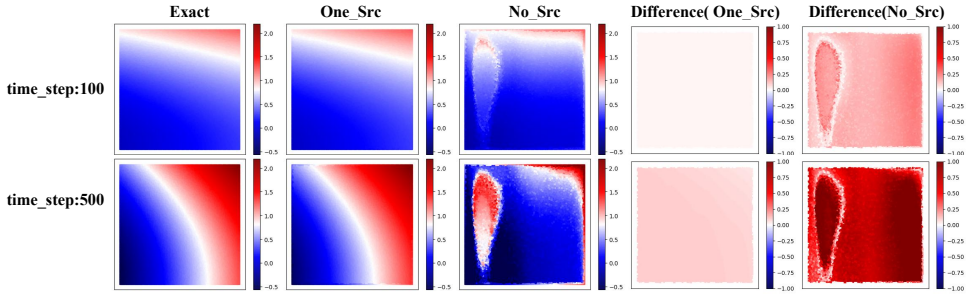


FIGURE 21. Generalization performance of the heat equation with different choices of node features.

TABLE 1. General comparison between PIGNN and PINN.

	PIGNN	PINN
number of training samples	0	0
mesh	regular/irregular	meshless
mesh size	flexible	—
problem scale	big	small
inference error	10^{-2}	10^{-2}
extension of computational domain	extend significantly to huge domain	extend moderately
extension of time steps	extend plenty of time steps	—
change PDE parameters	YES	NO
inverse problem	YES	YES
conclusion	Small area training, large area application.	training area $\approx$ application area

of graph neural networks and physics-informed loss function, the proposed PIGNN can naturally handle irregular meshes that can fit any geometry, as well as learn dynamics of the underlying system. We demonstrate the effectiveness and excellent performance of our PIGNN in terms of accuracy, time extrapolability and generalizability through a series of numerical experiments. Promising numerical results show that PIGNN outperforms the neural-based solver PINN. Most importantly, PIGNN only needs to be trained in small domains under simple settings to have excellent fitting ability, and then it can be directly applied to more complex situations with large domains, which further sheds lights on the superiority of the proposed model. Moreover, we show that PIGNN can scale well to distributed system to handle large-scale problems. The outstanding performance of PIGNN suggests that it has promising potential to serve as a reliable surrogate model to learn physical dynamics described by PDEs or as a neural-based solver to infer solutions for PDEs. Furthermore, our work reveals that integrating deep learning techniques, scientific knowledge and numerical methods together can facilitate the entire modelling and learning process of complex systems.

Acknowledgements.

Zhang Hao laid the mathematical foundation of the project, designed the main algorithms and supervised the entire project. Jiang Longxiang implemented, trained and tested most of the code and conducted some experiments. Wang Liyuan wrote and revised the whole manuscript, optimized the training process and completed most of the numerical experiments. Chu Xinkun revised the writing of the whole manuscript and provided support to the design of some experiments. Wen Yong and Li Luxiong helped to do some numerical experiments. Xiao Yonghao helped with arrangements of the project. Thanks for all the efforts we have done to accomplish this paper. The present research was supported by Sichuan Province Science Foundation for Youths under Grant Number 2023NSFSC1419.

Appendix.

A. Proof of Lemma 3.2.

Proof. Let l_i be the order of φ_i $(i = 1, \cdots, n)$, i.e., $\varphi_i = x^s y^{l_i - s}$, for some $s \in [0, l_i]$, then we have

$$\varphi_i(r\mathbf{x}_k) = r^{l_i}\varphi_i(\mathbf{x}_k),\ \forall\ k \in 1, \cdots, p. \tag{26}$$

Partitioning matrix $\mathbf{\Phi}$ according to the order of φ_i, i.e., $\mathbf{\Phi} = \left(\mathbf{\Phi}_1^{\mathrm{T}}, \cdots, \mathbf{\Phi}_m^{\mathrm{T}}\right)^{\mathrm{T}}$, where $\mathbf{\Phi}_j$ $(j \in 1, \cdots, m)$ is the submatrix of $\mathbf{\Phi}$ formed by taking all rows of entries φ_i with order of j, partitioning vector $\mathbf{Y}$ accordingly. By the definition of $\mathbf{Y}$ and φ_i, $\boldsymbol{Y}_i \neq \mathbf{0}$ only for $i = 2$.

Let $\boldsymbol{\eta} = r^2\boldsymbol{\omega}$, by Eq. (26), (19) can be written as

$$\mathbf{Y} = \mathbf{\Lambda}_r \mathbf{\Phi} \boldsymbol{\eta}, \tag{27}$$

where $\mathbf{\Lambda}_r = \mathrm{diag}(r^{-1}, 1, r, \cdots, r^{m-2})$.

Let $\boldsymbol{\eta}_r \in \mathbb{R}^p$ be the least square solution of Eq. (27).

1) Let us first study that $\boldsymbol{\eta}_r$ is bounded.

By Eq. (27),

$$\|\mathbf{Y} - \mathbf{\Lambda}_r\mathbf{\Phi}\boldsymbol{\eta}\|^2 = r^{-2}\|\mathbf{\Phi}_1\boldsymbol{\eta}\|^2 + \|\mathbf{Y}_2 - \mathbf{\Phi}_2\boldsymbol{\eta}\|^2 + \cdots + r^{2(m-2)}\|\mathbf{\Phi}_m\boldsymbol{\eta}\|^2. \tag{28}$$

Suppose $\boldsymbol{\eta}_0$ is an approximation of Laplacian Δ with the largest order of l. According to definition 3.1, $\mathbf{Y}_i - \mathbf{\Phi}_i\boldsymbol{\eta}_0 = \mathbf{0}$, for $i = 1, 2, \cdots, l,$, while $\mathbf{Y}_{l+1} - \mathbf{\Phi}_{l+1}\boldsymbol{\eta}_0 \neq \mathbf{0}$.

Since $\boldsymbol{\eta}_r$ is the least square solution of Eq. (27), we have

$$\|\mathbf{Y} - \mathbf{\Lambda}_r\mathbf{\Phi}\boldsymbol{\eta}_r\|^2 \leq \|\mathbf{Y} - \mathbf{\Lambda}_r\mathbf{\Phi}\boldsymbol{\eta}_0\|^2, \tag{29}$$

It follows from (28) and (29) that

$$r^{2(i-2)}\|\mathbf{\Phi}_i\boldsymbol{\eta}_r\|^2 \leq r^{2(l-1)}\|\mathbf{\Phi}_{l+1}\boldsymbol{\eta}_0\|^2 + \cdots + r^{2(m-2)}\|\mathbf{\Phi}_m\boldsymbol{\eta}\|^2, \forall\, i = 1, 2, \cdots, l+1.$$

This implies there exists a constant $C > 0$ such that for any small enough r, we have

$$\|\mathbf{\Phi}_i\boldsymbol{\eta}_r\|^2 \leq C\|\mathbf{\Phi}_{l+1}\boldsymbol{\eta}_0\|^2,\ \text{for } i = 1, 2, \cdots, l+1. \tag{30}$$

Let $\mathbf{\Phi}_{[l]} = \left[\mathbf{\Phi}_1^{\mathrm{T}}, \cdots, \mathbf{\Phi}_l^{\mathrm{T}}\right]^{\mathrm{T}}$, $\boldsymbol{Y}_{[l]} = [\boldsymbol{Y}_1^{\mathrm{T}}, \cdots, \boldsymbol{Y}_l^{\mathrm{T}}]^{\mathrm{T}}$. Since $\mathbf{\Phi}_{[l]}\boldsymbol{\eta} = \boldsymbol{Y}_{[l]}$ admits a solution, $\mathbf{\Phi}_{[l+1]}\boldsymbol{\eta} = \boldsymbol{Y}_{[l+1]}$ has no solution, $\mathbf{\Phi}_{[l+1]}$ is a full rank matrix.

Select the submatrix $\tilde{\mathbf{\Phi}}$ from $\mathbf{\Phi}_{[l+1]}$ such that $\tilde{\mathbf{\Phi}}$ is a nonsingular square matrix, and select the corresponding rows from $\boldsymbol{Y}_{[l+1]}$ to form $\tilde{\boldsymbol{Y}}$. It easily follows that $\boldsymbol{\eta}_r = \tilde{\mathbf{\Phi}}^{-1}\tilde{\boldsymbol{Y}}$.

Thus

$$\|\boldsymbol{\eta}_r\| \leq \|\tilde{\boldsymbol{\Phi}}^{-1}\|\|\tilde{\boldsymbol{Y}}\| \tag{31}$$

By Eq. (30), $\|\boldsymbol{Y}_{[l+1]}\| = \|\boldsymbol{\Phi}_{[l+1]}\boldsymbol{\eta}_r\|$ is bounded, which implies the right-hand side of Eq. (31) is bounded. Hence, assertion 1) follows.

2) Next, let $\boldsymbol{\eta}_n = \boldsymbol{\eta}_{r_n}$, and $\{\boldsymbol{\eta}_n\}_{n=1}^{\infty}$ be any convergent subsequence of $\boldsymbol{\eta}_r$, we examine that if $\boldsymbol{\eta} = \lim_{n\to\infty} \boldsymbol{\eta}_n$, then $\boldsymbol{\eta}$ is an approximation of Laplacian Δ with the largest order.

We argue this by contradiction. Let $\boldsymbol{\eta}_0$ be an approximation of Laplacian Δ with the largest order of l, suppose $\boldsymbol{\eta}$ is an approximation of Δ with order of s $(s < l)$.

By (28), we have

$$\|\mathbf{Y} - \boldsymbol{\Lambda}_r\boldsymbol{\Phi}\boldsymbol{\eta}\|^2 = r^{2(s-1)}\|\boldsymbol{\Phi}_{s+1}\boldsymbol{\eta}\|^2 + \cdots + r^{2(m-2)}\|\boldsymbol{\Phi}_m\boldsymbol{\eta}\|^2, \tag{32}$$

$$\|\mathbf{Y} - \boldsymbol{\Lambda}_r\boldsymbol{\Phi}\boldsymbol{\eta}_0\|^2 = r^{2(l-1)}\|\boldsymbol{\Phi}_{l+1}\boldsymbol{\eta}_0\|^2 + \cdots + r^{2(m-2)}\|\boldsymbol{\Phi}_m\boldsymbol{\eta}_0\|^2. \tag{33}$$

Noting (32), (33) and $\boldsymbol{\eta} = \lim_{n\to\infty} \boldsymbol{\eta}_n$, there exists constants $\delta > 0, \epsilon > 0$ and $M > 0$ such that

$$\begin{aligned}\|\mathbf{Y} - \boldsymbol{\Lambda}_{r_n}\boldsymbol{\Phi}\boldsymbol{\eta}_n\|^2 \geq r_n^{2(s-1)}\|\boldsymbol{\Phi}_{s+1}\boldsymbol{\eta}_n\|^2 &> \frac{r_n^{2(s-1)}}{2}\|\boldsymbol{\Phi}_{s+1}\boldsymbol{\eta}\|^2 \\ &\geq \varepsilon r_n^{2(s-1)} \geq M r_n^{2(l-1)} \geq \|\mathbf{Y} - \boldsymbol{\Lambda}_{r_n}\boldsymbol{\Phi}\boldsymbol{\eta}_0\|^2, \\ &\quad \text{for } n \text{ large enough.}\end{aligned}$$

This contradicts to the fact that $\boldsymbol{\eta}_n$ is the least square solution of Eq. (27).

□

B. Training setup.

B1. Setup of PIGNN.

The specific settings for each component of our PIGNN are as follows. In ENCODER, two MLPs are used to encode node features and edge features, respectively. PROCESSOR is composed of L GN blocks with the same structure, where L is set to 1 by default. Two MLPs are used as update functions for internal nodes features and edges features. DECODER is also implemented with a MLP. All these MLPs contain two hidden layers, each with 128 rectified linear unit (ReLU) activation units. By default, we set the time interval $\Delta t = 10^{-3}$, the chosen computational domain is $[0, 1] \times [0, 1]$ and is discretized by a 100×100 irregular mesh, for a total of 15681 nodes. We use the python API of FEniCS to automatically create the irregular triangular mesh over the computational domain. Fig. 22 shows an example of an irregular mesh on a regular domain $[0, 1] \times [0, 1]$ with a mesh width $\delta \mathrm{x} = 0.1$ (or mesh density 10) generated by FEniCS. By default, our model is trained for only 10 time steps using the Adam (Adaptive moment estimation) optimizer with an exponential learning rate decay strategy from 10^{-3} to 10^{-7} every 10 epochs by 1%.

B2. Setup of Baseline models.

With the help of DEEPXDE [38] library, we built the PINN baseline model. For a fair comparison, PINN's MLP architecture contains four hidden layers with 20 Tanh activation units each. The loss function is calculated by randomly selecting

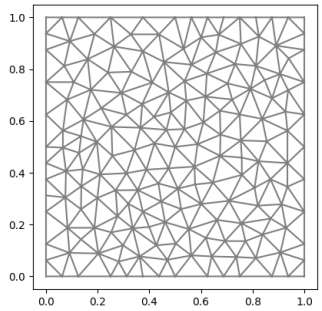


FIGURE 22. An irregular mesh (mesh density is 10) on a regular domain $[0, 1] \times [0, 1]$ generated by FEniCS.

collocation points from both spatial and time spaces, where the number of collocation points is approximately equal to the number of mesh points multiplied by the total training time steps. When training, the computational domain and the time domain are the same as those in our approach. We train the model in two stages. In the first stage, the network is trained by the Adam optimizer with a learning rate of 10^{-3} for 6×10^4 epochs, and then in the second stage, it is trained by the L-BFGS (Limited-memory BFGS) optimizer for additional 4×10^4 epochs.

B3. Selection of hyper-parameters.

An appropriate selection of hyper-parameters contributes to the success of model training. In this subsection, we introduce the criteria for selecting hyper-parameters.

- γ: This parameter balances the data loss and residual physics loss for network training in the inverse problem to help the network identify unknown PDE parameters. In practice, there is an order of magnitude difference between residual loss and data loss at the beginning of network training. Therefore, the initial value of γ is set large (e.g., 0.9-0.95) to reduce the magnitude difference while allowing the network to mainly learn the dynamics of PDEs during the initial training period. As the training process proceeds, we gradually reduce γ to 0.5 so that the network can learn more from data.
- Δt: This parameter represents the time interval of the time space discretization. By default, we set it to 10^{-3}. We use this setup in all experiments of Burgers and FN equations. For the heat equation, we set $\Delta t = 10^{-4}$ since we are unable to successfully train the model using the default settings. Fortunately, numerical studies have provided us with a stability criterion for the spatialtemporal resolution. According to this criterion, the magnitude of Δt is set to approximately the square of the spatial resolution, namely, $\Delta t \sim (\delta x)^2$, where δx is the mesh width which in our experiments is set to 10^{-2}.
- Training time steps: This parameter represents the number of time steps for network training. By default, we set it to 10, and use this setting in all experiments of Burgers and FN equations. For the heat equation, we set it to 50 because of its relatively small time interval. We set it larger to allow the model to learn more information over a longer evolutionary period.

- Maximum number of training epochs: Our principle for setting the number of training epochs is that the model should be sufficiently trained. Specifically, for different equations, we adaptively adjust the concrete value according to the complexity of the problem. Generally, the number of training epochs for the Adam optimizer is about $6 \times 10^4 \sim 7 \times 10^4$, while for the L-BFGS optimizer, $3 \times 10^4 \sim 4 \times 10^4$ training epochs are used.

C. PDE examples.

The specific expressions of PDE examples are as follows.

1) 2D Heat equation

$$\begin{aligned} \frac{\partial u(x,y,t)}{\partial t} &= \Delta u(x,y,t) + f(x,y,t), \qquad (34)\\ f(x,y,t) &= a\cos[b\pi(x-d)t + cy^2][b\pi(x-d) - 2c] \\ &\quad + a\sin[b\pi(x-d)t + cy^2][b^2\pi^2t^2 + 4c^2y^2], \\ \Delta u(x,y,t) &= \frac{\partial^2 u(x,y,t)}{\partial x^2} + \frac{\partial^2 u(x,y,t)}{\partial y^2}, \\ (x,y) \in \Omega &= [0,1]\times[0,1],\ t > 0, \end{aligned}$$

subject to the initial and boundary conditions

$$\begin{aligned} u(x,y,0) &= a\sin(cy^2), && (x,y) \in \Omega, \\ u(x,y,t) &= a\sin[b\pi(x-d)t + cy^2], && (x,y) \in \partial\Omega,\ t > 0. \end{aligned}$$

The analytical solution is

$$u(x,y,t) = a\sin[b\pi(x-d)t + cy^2], \qquad (35)$$

where a, b, c, $d = 4$, 2, 0.5, 0.3.

2) 2D Burgers equation

$$\begin{aligned} \frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} &= \frac{1}{R_e}\Delta u, \qquad (36)\\ \frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} &= \frac{1}{R_e}\Delta v, \qquad (37)\\ (x,y) \in \Omega &= [0,1]\times[0,1],\ t > 0, \end{aligned}$$

where $R_e = 80$ is the Reynolds number, the analytical solutions below at $t = 0$ specifies the initial conditions, and the boundary conditions are also specified by the following analytical solutions.

$$\begin{aligned} u(x,y,t) &= \frac{3}{4} - \frac{1}{4[1+\exp((-t-4x+4y)R_e/32)]}, \qquad (38)\\ v(x,y,t) &= \frac{3}{4} + \frac{1}{4[1+\exp((-t-4x+4y)R_e/32)]}. \qquad (39) \end{aligned}$$

3) 2D FitzHugh-Nagumo reaction-diffusion equation

$$\begin{aligned} \frac{\partial u}{\partial t} &= \gamma_u\Delta u + u - u^3 - v + \alpha, \qquad (40)\\ \frac{\partial v}{\partial t} &= \gamma_v\Delta v + \beta(u - v), \qquad (41)\\ (x,y) \in \Omega &= [0,1]\times[0,1],\ t > 0, \end{aligned}$$

where $\gamma_u = \gamma_v = 0.001, \alpha = 0, \beta = 1$.

subject to the boundary conditions

$$u(x,y,t)=0,\ v(x,y,t)=0,\ (x,y)\in\partial\Omega, t>0,$$

and the initial conditions

$$u(x,y,0) = \sin(b\pi x)\cos[a\pi(y-d)]e^{-c(x^2+y^2)}, \quad (42)$$

$$v(x,y,0) = \cos[a\pi(x-d)]\sin(b\pi y)e^{-c(x^2+y^2)}, \quad (43)$$

where a, b, c, $d = 1,\ 1,\ 6,\ 0$.

REFERENCES

[1] Raissi M, Perdikaris P, Karniadakis G E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations[J]. Journal of Computational physics, 2019, 378: 686-707.

[2] Cai S, Mao Z, Wang Z, et al. Physics-informed neural networks (PINNs) for fluid mechanics: A review[J]. Acta Mechanica Sinica, 2022: 1-12.

[3] Wong J C, Ooi C, Chiu P H, et al. Improved Surrogate Modeling of Fluid Dynamics with Physics-Informed Neural Networks[J]. arXiv preprint arXiv:2105.01838, 2021.

[4] He Q Z, Barajas-Solano D, Tartakovsky G, et al. Physics-informed neural networks for multiphysics data assimilation with application to subsurface transport[J]. Advances in Water Resources, 2020, 141: 103610.

[5] Mao Z, Jagtap A D, Karniadakis G E. Physics-informed neural networks for high-speed flows[J]. Computer Methods in Applied Mechanics and Engineering, 2020, 360: 112789.

[6] Arzani A, Wang J X, D'Souza R M. Uncovering near-wall blood flow from sparse data with physics-informed neural networks[J]. Physics of Fluids, 2021, 33(7): 071905.

[7] Sahli Costabal F, Yang Y, Perdikaris P, et al. Physics-informed neural networks for cardiac activation mapping[J]. Frontiers in Physics, 2020, 8: 42.

[8] Yin M, Zheng X, Humphrey J D, et al. Non-invasive inference of thrombus material properties with physics-informed neural networks[J]. Computer Methods in Applied Mechanics and Engineering, 2021, 375: 113603.

[9] Zhang E, Yin M, Karniadakis G E. Physics-informed neural networks for nonhomogeneous material identification in elasticity imaging[J]. arXiv preprint arXiv:2009.04525, 2020.

[10] Haghighat E, Raissi M, Moure A, et al. A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics[J]. Computer Methods in Applied Mechanics and Engineering, 2021, 379: 113741.

[11] Zhang R, Liu Y, Sun H. Physics-informed multi-LSTM networks for metamodeling of nonlinear structures[J]. Computer Methods in Applied Mechanics and Engineering, 2020, 369: 113226.

[12] Kharazmi E, Zhang Z, Karniadakis G E. Variational physics-informed neural networks for solving partial differential equations[J]. arXiv preprint arXiv:1912.00873, 2019.

[13] Sitzmann V, Martel J, Bergman A, et al. Implicit neural representations with periodic activation functions[J]. Advances in Neural Information Processing Systems, 2020, 33: 7462-7473.

[14] Wight C L, Zhao J. Solving allen-cahn and cahn-hilliard equations using the adaptive physics informed neural networks[J]. arXiv preprint arXiv:2007.04542, 2020.

[15] Nabian M A, Gladstone R J, Meidani H. Efficient training of physics-informed neural networks via importance sampling[J]. Computer-Aided Civil and Infrastructure Engineering, 2021, 36(8): 962-977.

[16] Pang G, Lu L, Karniadakis G E. fPINNs: Fractional physics-informed neural networks[J]. SIAM Journal on Scientific Computing, 2019, 41(4): A2603-A2626.

[17] McClenny L, Braga-Neto U. Self-adaptive physics-informed neural networks using a soft attention mechanism[J]. arXiv preprint arXiv:2009.04544, 2020.

[18] Xiang Z, Peng W, Zheng X, et al. Self-adaptive loss balanced physics-informed neural networks for the incompressible navier-stokes equations[J]. arXiv preprint arXiv:2104.06217, 2021.

[19] Gao H, Sun L, Wang J X. PhyGeoNet: physics-informed geometry-adaptive convolutional neural networks for solving parameterized steady-state PDEs on irregular domain[J]. Journal of Computational Physics, 2021, 428: 110079.

[20] Rodriguez-Torrado R, Ruiz P, Cueto-Felgueroso L, et al. Physics-informed attention-based neural network for solving non-linear partial differential equations[J]. arXiv preprint arXiv:2105.07898, 2021.
[21] Ren P, Rao C, Liu Y, et al. PhyCRNet: Physics-informed convolutional-recurrent network for solving spatiotemporal PDEs[J]. Computer Methods in Applied Mechanics and Engineering, 2022, 389: 114399.
[22] Zhang D, Lu L, Guo L, et al. Quantifying total uncertainty in physics-informed neural networks for solving forward and inverse stochastic problems[J]. Journal of Computational Physics, 2019, 397: 108850.
[23] Chen Z, Liu Y, Sun H. Physics-informed learning of governing equations from scarce data[J]. Nature communications, 2021, 12(1): 1-13.
[24] Meng X, Yang L, Mao Z, et al. Learning functional priors and posteriors from data and physics[J]. Journal of Computational Physics, 2022, 457: 111073.
[25] Sanchez-Gonzalez A, Godwin J, Pfaff T, et al. Learning to simulate complex physics with graph networks[C]//International Conference on Machine Learning. PMLR, 2020: 8459-8468.
[26] Sanchez-Gonzalez A, Godwin J, Pfaff T, et al. Learning to simulate complex physics with graph networks[C]//International Conference on Machine Learning. PMLR, 2020: 8459-8468.
[27] Jiang L, Wang L, Chu X, et al. PhyGNNet: Solving spatiotemporal PDEs with Physics-informed Graph Neural Network[C]//Proceedings of the 2023 2nd Asia Conference on Algorithms, Computing and Machine Learning. 2023: 143-147.
[28] Gilmer J, Schoenholz S S, Riley P F, et al. Neural message passing for quantum chemistry[C]//International conference on machine learning. PMLR, 2017: 1263-1272.
[29] Taubin G. A signal processing approach to fair surface design[C]//Proceedings of the 22nd annual conference on Computer graphics and interactive techniques. 1995: 351-358.
[30] Taubin G. Geometric signal processing on polygonal meshes[C]//Proceedings of EUROGRAPHICS. 2000.
[31] Fujiwara K. Eigenvalues of laplacians on a closed riemannian manifold and its nets[J]. Proceedings of the American Mathematical Society, 1995, 123(8): 2585-2594.
[32] Desbrun M, Meyer M, Schröder P, et al. Implicit fairing of irregular meshes using diffusion and curvature flow[C]//Proceedings of the 26th annual conference on Computer graphics and interactive techniques. 1999: 317-324.
[33] Mayer U F. Numerical solutions for the surface diffusion flow in three space dimensions[J]. Computational and Applied Mathematics, 2001, 20(3): 361-379.
[34] Meyer M, Desbrun M, Schröder P, et al. Discrete differential-geometry operators for triangulated 2-manifolds[M]//Visualization and mathematics III. Springer, Berlin, Heidelberg, 2003: 35-57.
[35] Wardetzky M, Mathur S, Kälberer F, et al. Discrete Laplace operators: no free lunch[C]//Proceedings of Eurographics Symposium on Geometry Processing. 2007: 33-37.
[36] Burgers J M. A mathematical model illustrating the theory of turbulence[J]. Advances in applied mechanics, 1948, 1: 171-199.
[37] FitzHugh R. Mathematical models of threshold phenomena in the nerve membrane[J]. The bulletin of mathematical biophysics, 1955, 17(4): 257-278.
[38] Lu L, Meng X, Mao Z, et al. DeepXDE: A deep learning library for solving differential equations[J]. SIAM Review, 2021, 63(1): 208-228.